

Detalhes

Fechar

Confirmar

Cancelar

Aceito

Não Aceito



Bloco de Notas

Salvar

Cancelar

Imprimir

Dicionário

Fechar

Pesquisa

BDTD

SciELO

Google

<http://bdtb.ibict.br/vufind/>

Fechar



Envie sua dúvida no Correio

Título

Dúvida

Cancelar

Enviar

Canal de Relacionamento

Ajuda (<https://md.claretiano.edu.br/boapraest-fev-2022-ead/>)

Para utilizar nosso serviço de atendimento, siga as instruções abaixo:

- Acesse o link: <http://atendimento.claretiano.edu.br/login> (<http://atendimento.claretiano.edu.br/login>).
- Informe seu CPF e data de nascimento.
- Na próxima tela, você terá acesso aos atendimentos já abertos e poderá interagir conforme necessário.
- Para abrir um novo pedido de suporte, clique na opção “Novo Atendimento”, localizada na parte inferior da tela. Escolha a modalidade e o setor desejado.
- Após o registro, nossa central de atendimento dará continuidade ao seu pedido, e você será notificado por e-mail sempre que houver alguma atualização.



Lembre-se de salvar o link do novo processo de atendimento em seus favoritos: <http://atendimento.claretiano.edu.br/login> (<http://atendimento.claretiano.edu.br/login>).

Fechar



Fechar

Sumário

Digite o que deseja encontrar nesta página



 Assistente Virtual

 Clique para Habilitar/Desabilitar o marcador

 Abrir o bloco de notas

 Informe sua dúvida sobre o conteúdo

 Pesquise sobre o assunto na internet

 Busque o termo no dicionário

 Veja sua trilha de estudos

 Apagar ou acender a luz de fundo

 Imprima o conteúdo da página estudada

 Gerar material em PDF

 Opções de Acessibilidade

 Canal de Relacionamento



  (https://md.claretiano.edu.br/
algpro-gs0067-fev-2023-grad-ead/)

1. Introdução

Neste material, você terá o primeiro contato com os principais conceitos relacionados à programação de computadores. Que tal uma ideia inicial sobre isso? Então, assista ao vídeo a seguir:



Para facilitar o seu estudo, o conteúdo foi dividido em cinco unidades. Nelas, você terá a oportunidade de aprender como elaborar algoritmos capazes de solucionar não só os problemas computacionais, mas também os do cotidiano, como o que relatamos no vídeo.

Estudará, ainda, tópicos fundamentais para a construção dos algoritmos, explicitando desde os tipos de dados até as estruturas sequenciais e de repetição, passando, inclusive, pelas variáveis. Além disso, você poderá estudar as estruturas de dados para armazenamento de grandes conjuntos de informações, como, por exemplo, as variáveis compostas homogêneas e as variáveis compostas heterogêneas.

Por fim, você terá a possibilidade de aprender como organizar eficientemente seus algoritmos por meio de técnicas de programação modular.

Dessa forma, é essencial que, para um bom aproveitamento de seus estudos, você pratique. Para isso, não deixe de realizar os exercícios propostos. Não se esqueça de que também é fundamental que você compartilhe seus conhecimentos na Sala de Aula Virtual.

Uma vez que este material é apenas uma referência, é importante que você complemente seu estudo por meio de outras fontes de informações, como livros, revistas e internet.

Bons estudos!

2. Informações da Disciplina



Ementa

Programar um computador significa escrever soluções para problemas reais em forma de algoritmos que possam ser interpretados e executados por computadores. Para adquirir tais habilidades, esta disciplina apresenta os princípios da programação de computadores; lógica booleana; conceitos de algoritmos e suas representações; variáveis e tipos de dado; estrutura sequencial, estruturas condicionais e estruturas de repetição; estruturas de dados homogêneas (vetores e matrizes) e heterogêneas (registros); programação modular.

Objetivo Geral

Os alunos da disciplina **Algoritmos e Programação** serão capazes de compreender e se familiarizar com os aspectos teóricos e práticos do projeto e construção de algoritmos. Nesse contexto, terão a possibilidade de estudar os principais aspectos relacionados à elaboração de algoritmos, os quais são fundamentais para o desenvolvimento de qualquer tipo de sistema computacional, independentemente

da linguagem de programação utilizada.

Objetivos Específicos



- Conhecer os principais conceitos relacionados à programação.
- Conhecer os principais tópicos para a construção dos algoritmos, desde os tipos de dados até as estruturas sequenciais e de repetição, passando, inclusive, pelas variáveis.
- Compreender as estruturas de dados para armazenamento de grandes conjuntos de informações, como, por exemplo, as variáveis compostas homogêneas.
- Aprender como organizar, de maneira eficiente, seus algoritmos por meio de técnicas de programação.



Unidade 1 – Princípios da Programação de Computadores

Objetivos

- Compreender os principais conceitos relativos à lógica matemática.
- Elaborar e representar algoritmos.

Conteúdos

- Fundamentos da lógica.
- Elaboração e representação de algoritmos.

Problematização

Qual a importância do uso da lógica para a elaboração de algoritmos? Como elaborar e representar algoritmos concisos?



Orientações para o estudo

Durante o estudo desta unidade, você deverá ler além dos conteúdos aqui apresentados, bem como assistir aos vídeos propostos para ampliar seus conhecimentos relacionados aos assuntos. A leitura dos textos indicados (artigos e capítulos de livros) é de suma importância para a compreensão dos conceitos de lógica matemática apresentados durante a disciplina.

Boa leitura!

1. Introdução

Nesta primeira unidade, você terá a oportunidade de conhecer os conceitos fundamentais para a construção e representação de algoritmos.

Inicialmente, serão descritos os fundamentos da lógica de programação e suas principais terminologias. É fundamental que você se dedique aos seus estudos, pois a lógica de programação será essencial para a compreensão e análise dos algoritmos e programas.

Estudaremos, em seguida, os conceitos preliminares a respeito dos algoritmos e as principais técnicas de elaboração e representação.

Bons estudos!

2. Introdução à lógica

Provavelmente, você já utilizou a palavra **lógica** para referenciar algum tipo de evento ocorrido no seu dia a dia, como, por exemplo:

- É lógico que hoje vai chover.
- Se tudo correr bem, é lógico que vou ao cinema.
- Como você estudou muito para a prova, é lógico que acertará todas as questões.



No entanto, como podemos definir a palavra **lógica**?

Na maioria das vezes, a palavra **lógica** está relacionada ao pensamento, porém não a pensamentos desordenados ou confusos, mas a pensamentos coerentes e racionais. Para Aristóteles, considerado o pai da **lógica**, ela representa as leis do pensamento (GERSTING, 2004).

Assim, em todas as nossas atividades, precisamos utilizar a **lógica**. Quando, por exemplo, estamos escrevendo uma mensagem eletrônica ou falando ao telefone, precisamos dela para ordenar nossos pensamentos.

Também utilizamos a **lógica** para agir corretamente. Imagine que você precisa sair da sua casa e ir ao supermercado comprar leite. Apesar de realizar essa tarefa facilmente, você, inconscientemente, utiliza conceitos lógicos para determinar quais passos ou quais caminhos deverão ser seguidos até chegar ao seu destino (o supermercado). Caso contrário, você poderia ficar "andando em

círculos" e nunca encontrar o supermercado. Além disso, você ficaria sem o leite.

Você sabia que, para realizar tarefas mais simples, também utilizamos o raciocínio lógico?

Como exemplo, suponha que você deseja preparar um leite com achocolatado. Uma possível ordem lógica para a solução do problema seria:

Algoritmo 1 – preparar um leite com achocolatado

- 1 Abrir a geladeira.
- 2 Retirar o litro de leite.
- 3 Abrir o armário.
- 4 Retirar o pote de achocolatado.
- 5 Pegar um copo vazio.
- 6 Adicionar leite no copo.
- 7 Adicionar achocolatado no copo com o leite.
- 8 Misturar.

Apesar de ser considerada uma tarefa extremamente simples, veja qu



Outro fato importante que deve ser destacado é a ordem dos acontecimentos. A correta organização do raciocínio faz com que o resultado seja facilmente alcançado. Em contrapartida, imagine o seguinte raciocínio, desordenado logicamente:

Algoritmo 2 – preparar um leite com achocolatado

- 1 Pegar um copo vazio.
- 2 Retirar o litro de leite.
- 3 Abrir a geladeira.
- 4 Misturar.
- 5 Retirar o pote de achocolatado.
- 6 Adicionar leite no copo.
- 7 Abrir o armário.
- 8 Adicionar achocolatado no copo com o leite.

Observe que os passos são exatamente os mesmos. No entanto, estão de maneira desordenada. Dessa forma, seria difícil alcançarmos o objetivo proposto.

Neste momento, você provavelmente deve estar se perguntando: o que isso tem a ver com algoritmos e programação? É simples. A lógica auxiliará você na compreensão dos conceitos fundamentais para a construção de algoritmos. Além disso, com ela, você estará preparado para analisar a formalidade e a coerência dos programas.

Inicialmente, você terá a possibilidade de estudar as proposições lógicas, que são essenciais para a construção do raciocínio lógico organizado. Em seguida, aprenderá os conectivos lógicos, responsáveis pelo agrupamento das proposições. E, finalmente, poderá explorar uma ferramenta importante para a lógica matemática, que é conhecida como **tabela-verdade**. Vamos iniciar nossos estudos pela abordagem das proposições.

Proposições

Uma das aplicações da lógica é a representação simbólica de raciocínios. Desse modo, podemos dizer que as afirmações que realizamos diariamente são transformadas em símbolos para demonstrar fatos ou transmitir informações.



Uma proposição representa uma afirmação (ou sentença), que pode assumir o valor lógico **verdadeiro** ou o valor **falso**. As proposições são sempre simbolizadas por letras minúsculas {a, b, c [...] x, y, z}, conforme podem ser observadas na Tabela 1.

Tabela 1 Valores lógicos.

Sentença	Valor Lógico	Símbolo
As bolas de futebol são redondas.	Verdadeiro	p
O sol é azul.	Falso	q
Dez é menor que sete.	Falso	r
A neve é branca.	Verdadeiro	s

Fonte: desenvolvida pelos autores.

Atenção!

Há sentenças que não possuem valor lógico e, por isso, não são consideradas proposições. Por exemplo:

- Como está você?
- Ela é muito talentosa.
- Existe vida em outros planetas.

Com base nesta explicação, analise se compreendeu, de fato, o que é proposição. Para isso, responda à questão a seguir:

Nos próximos subtópicos, vamos entender o que são os conectivos lógicos e, na sequência, a tabela-verdade.

Conectivos lógicos

Os conectivos lógicos são símbolos utilizados para combinar proposições. No dia a dia, utilizamos esses conectivos ao falar, ao escrever. As proposições formadas por meio da combinação de conectivos são chamadas **proposições compostas** (GERSTING, 2004).



Observe o exemplo na Tabela 2:

Tabela 2 Proposições.

Sentença	Valor Lógico	Símbolo
Os cachorros possuem quatro patas.	Verdadeiro	p
Os gatos adoram leite.	Verdadeiro	q

Fonte: desenvolvida pelos autores.

Utilizando as proposições simples (p, q), podemos formar algumas proposições compostas com os conectivos **e**, **ou**, **se [...] então**. Observe como são construídas essas proposições na Tabela 3.

Tabela 3 Proposições Compostas.

Proposição	Conectivo	Proposição	Proposição Composta
------------	-----------	------------	---------------------

P	e	q	Os cachorros possuem quatro patas e os gatos adoram leite.
P	ou	q	Os cachorros possuem quatro patas ou os gatos adoram leite.
P	se ... então	q	Se os cachorros possuem quatro patas, então os gatos adoram leite.

Fonte: desenvolvida pelos autores.

É importante notar que o valor lógico de uma proposição composta depende dos valores lógicos das proposições simples que a compõem.

Na lógica matemática, cada conectivo é representado por um símbolo, conforme é apresentado na Tabela 4.

Tabela 4 Conectivos.

Conectivo Lógico	Significado	Símbolo
Negação	não	$\sim$
Conjunção	e	$\wedge$
Disjunção	ou	$\vee$
Disjunção Exclusiva	ou exclusivo	$\underline{\vee}$
Condicional	se ... então	$\rightarrow$
Bicondicional	se, somente se	$\leftrightarrow$



Fonte: desenvolvida pelos autores.

Na formulação de proposições compostas, podemos utilizar diversos conectivos, porém lembre-se de que é necessário inserir um conectivo entre as duas proposições, conforme o exemplo:

- $(a \ b) \rightarrow C \rightarrow$ incorreto, falta um conectivo entre **a** e **b**;
- $(a \ \cup \ b) \rightarrow C \rightarrow$ correto.

Tabelas-verdade

As tabelas-verdade são um recurso muito utilizado na avaliação de proposi-

ções lógicas. Elas permitem avaliar uma proposição em função de todas as entradas possíveis.

Desse modo, o número de linhas de uma tabela-verdade depende do número de proposições simples da proposição composta, assumindo sempre a seguinte regra: 2^n (lê-se **dois** elevado a **n**), em que **n** representa o número de proposições simples.

Alternativamente, podemos considerar as proposições simples como variáveis e as proposições compostas como expressões. Veja um exemplo na Tabela 5.

Tabela 5 Variáveis e expressões.

Expressões	Número de variáveis	Número de linhas da tabela verdade (2^n)
$(a \wedge b)$	2	$2^2 = 4$
$(a \wedge b) \leftrightarrow c$	3	$2^3 = 8$
$(a \wedge b) \leftrightarrow (c \vee d)$	4	$2^4 = 16$

Fonte: desenvolvida pelos autores.

Para construir a tabela-verdade, é necessário avaliar todas as possibilidades de entradas para cada proposição simples. Observe, na Tabela 6, uma possível tabela-verdade para a proposição $(a \dot{\cup} b)$:

Tabela 6 Tabela-verdade para a proposição $(a \dot{\cup} b)$.

	Entrada		Saída
	a	b	...
1	Verdadeiro	Verdadeiro	...
2	Verdadeiro	Falso	...
3	Falso	Verdadeiro	...
4	Falso	Falso	...

Para simplificar, podemos representar o valor lógico verdadeiro como **V** e o valor lógico falso como **F**. Dessa forma, para a proposição composta $(a \cup b) \rightarrow c$, temos a tabela-verdade descrita na Tabela 7.

Tabela 7 Tabela-verdade da proposição composta $(a \cup b) \rightarrow c$.

	Entrada			Saída
	a	b	c	...
1	V	V	V	...
2	V	V	F	...
3	V	F	V	...
4	V	F	F	...
5	F	V	V	...
6	F	V	F	...
7	F	F	V	...
8	F	F	F	...



Fonte: desenvolvida pelos autores.

Cada conectivo lógico possui sua tabela-verdade. Assim, a tabela-verdade do conectivo lógico **negação**, simbolizado por $\sim p$ (leia-se "**n**ão **p**"), é representada conforme a Tabela 8.

Tabela 8 Tabela-verdade do conectivo $\sim p$.

p	$\sim p$
V	F
F	V

Fonte: desenvolvida pelos autores.

Em que **p** e **~p** representam as seguintes sentenças:

- **p** = Hoje está frio.
- **~p** = Hoje não está frio.



Vamos entender um pouco mais sobre os conectivos lógicos?

Para que você compreenda melhor os princípios de conjunção e disjunção que abordaremos a seguir, recomendamos que assista à videoaula *Raciocínio Lógico – Aula 2 – Conectivos e Tabela-verdade* (<https://youtu.be/L73yZkTQ9pE>).

As proposições compostas do tipo **p** **∩** **q** são chamadas de conjunção, que é entendida como **p** e **q**, e sua tabela-verdade é representada na forma encontrada na Tabela 9.

Tabela 9 Tabela-verdade da conjunção.



p	q	p ∩ q
V	V	V
V	F	F
F	V	F
F	F	F

Fonte: desenvolvida pelos autores.

Em que **p**, **q** e **p** **∪** **q** definem as seguintes sentenças:

- **p** = Fernanda é estudiosa.
- **q** = Fernanda tem um bom rendimento nas provas.
- **p** **∪** **q** = Fernanda é estudiosa e tem um bom rendimento nas provas.

A disjunção representa a combinação **ou**, expressa pelo símbolo **∪** (leia-se "**p** ou **q**"), e sua tabela-verdade está representada na Tabela 10.

Tabela 10 Tabela-verdade da disjunção.

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	F

Fonte: desenvolvida pelos autores.

De acordo com a Tabela 10, temos que:

- **p** = Hoje vou ao clube.
- **q** = Hoje vou trabalhar.
- **$p \vee q$** = Hoje vou ao clube **ou** hoje vou trabalhar.

A disjunção exclusiva, também conhecida como **ou exclusivo**, é representada pelo símbolo $\underline{\vee}$, e a tabela-verdade é demonstrada conforme a Tabela 11. Veja:



Tabela 11 Tabela-verdade da disjunção exclusiva.

p	q	$p \underline{\vee} q$
V	V	F
V	F	V
F	V	V
F	F	F

Fonte: desenvolvida pelos autores.

As seguintes sentenças apresentadas pelos símbolos são:

- **p** = Lúcia é bonita.
- **q** = Lúcia é inteligente.
- **$p \underline{\vee} q$** = **Ou** Lúcia é bonita ou é inteligente.

As proposições também podem ser combinadas na forma "se proposição **p**, então proposição **q**", e são representadas pelo símbolo $\rightarrow$. A proposição composta é denotada por **$p \rightarrow q$** (leia-se "**p implica q**"), e o seu conectivo é chamado **condicional**, ou **implicação**. Observe, na Tabela 12, a tabela-verdade expressa na forma condicional.

Tabela 12 Tabela-verdade do condicional.

p	q	$p \rightarrow q$
V	V	V
V	F	F
F	V	V
F	F	V

Fonte: desenvolvida pelos autores.

Em que temos as seguintes sentenças:

- **p** = Hoje é sábado.
- **q** = Hoje vou sair.
- **$p \rightarrow q$** = Se hoje é sábado, **então** vou sair.



O conectivo **bicondicional**, ou **equivalência**, é simbolizado por $\leftrightarrow$. A expressão **$p \leftrightarrow q$** é uma abreviatura de **$(p \rightarrow q) \wedge (q \rightarrow p)$** ; assim, a tabela-verdade pode ser definida como é mostrado na Tabela 13:

Tabela 13 Tabela-verdade do bicondicional.

p	q	$p \leftrightarrow q$
V	V	V
V	F	F
F	V	F
F	F	V

Fonte: desenvolvida pelos autores.

E os símbolos da tabela representam as seguintes sentenças:

- **p** = João ganhará presentes no Natal.
- **q** = João é um bom menino.
- **p ↔ q** = João ganhará presentes no Natal **se, e somente se**, for um bom menino.

Veja, resumidamente, na Tabela 14, as tabelas-verdade para os principais conectivos.

Tabela 14 Tabela do resumo das tabelas-verdade dos principais conectivos.

p	q	p ∧ q	p ∨ q	p ⊆ q	p → q	p ↔ q
V	V	V	V	F	V	V
V	F	F	V	V	F	F
F	V	F	V	V	V	F
F	F	F	F	F	V	V

Fonte: desenvolvida pelos autores.

Conforme comentado anteriormente, podemos utilizar as tabelas-verdade para avaliar proposições compostas. Desse modo, vamos analisar alguns exemplos:

Exemplo 1: $(a \vee b) \rightarrow a$

Tabela 15 Tabela-verdade da proposição $(a \vee b) \rightarrow a$.

	Entrada			
	a	b	(a ∨ b)	① → a
1	V	V	V	V
2	V	F	V	V
3	F	V	V	F
4	F	F	F	V
			①	Resultado

Fonte: desenvolvida pelos autores.

No Exemplo 1, você pôde notar que as colunas 1 e 2 representam as possíveis

entradas para as proposições **a** e **b**. A coluna 3 representa o resultado da avaliação da proposição composta ($a \cup b$). Para facilitar o entendimento, a coluna 3 foi nomeada para .

Por fim, a coluna 4 contém o resultado da coluna 3 () combinada com o conectivo ® e a coluna 1. É sempre importante destacar a coluna que representa o resultado final da tabela-verdade. Além disso, você pôde observar que essa tabela possui um total de quatro linhas, pois o número de proposições (ou variáveis) é igual a 2, ou seja, $2^2 = 4$.

Exemplo 2: $(a \leftrightarrow b) \underline{\vee} (a \leftrightarrow c)$

Tabela 16 Tabela-verdade da proposição $(a \leftrightarrow b) \underline{\vee} (a \leftrightarrow c)$.

Entrada					
a	b	c	$(a \leftrightarrow b)$	① $\underline{\vee}$ ②	$(a \leftrightarrow c)$
V	V	V	V	F	V
V	V	F	V	V	F
V	F	V	F	V	V
V	F	F	F	F	F
F	V	V	F	F	F
F	V	F	F	V	V
F	F	V	V	V	F
F	F	F	V	F	V
			①	Resultado	②

Fonte: desenvolvida pelos autores.

No Exemplo 2, inicialmente, são resolvidas as expressões que estão dentro dos parênteses. O resultado da expressão $(a \leftrightarrow b)$ é expresso na coluna ①, enquanto o resultado da expressão $(a \leftrightarrow c)$ é representado na coluna ②. Finalmente, a coluna em destaque recebe a disjunção exclusiva entre as colunas ① e ②.

Exemplo 3: $\sim(a \vee b) \rightarrow ((b \wedge c) \leftrightarrow (b \wedge d))$

Tabela 17 Tabela-verdade da proposição $\sim(a \vee b) \rightarrow ((b \wedge c) \leftrightarrow (b \wedge d))$.

Entrada										
	a	b	c	d	$(a \vee b)$	$\sim ①$	$② \rightarrow ⑤$	$(b \wedge c)$	$③ \leftrightarrow ④$	$(b \wedge d)$
1	V	V	V	V	V	F	V	V	V	V
2	V	V	V	F	V	F	V	V	F	F
3	V	V	F	V	V	F	V	F	F	V
4	V	V	F	F	V	F	V	F	V	F
5	V	F	V	V	V	F	V	F	V	F
6	V	F	V	F	V	F	V	F	V	F
7	V	F	F	V	V	F	V	F	V	F
8	V	F	F	F	V	F	V	F	V	F
9	F	V	V	V	V	F	V	V	V	V
10	F	V	V	F	V	F	V	V	F	F
11	F	V	F	V	V	F	V	F	F	V
12	F	V	F	F	V	F	V	F	V	F
13	F	F	V	V	F	V	V	F	V	F
14	F	F	V	F	F	V	V	F	V	F
15	F	F	F	V	F	V	V	F	V	F
16	F	F	F	F	F	V	V	F	V	F
Resultado										

Fonte: desenvolvida pelos autores.



Agora que vimos exemplos de construção da tabela-verdade, é essencial compreendermos como seus resultados podem ser classificados, assunto que está exposto no tópico a seguir.

Tautologias, contradições e contingências

Os resultados das tabelas-verdade podem ser classificados considerando três critérios, a saber: tautologias, contradições e contingências. Observe, a seguir, a explicação de cada critério:

- **Tautologia:** quando o resultado da tabela-verdade apresenta apenas valores verdadeiros (V).
- **Contradição:** para tabelas-verdade contendo todos os valores resultantes falsos (F).
- **Contingência:** para tabelas-verdade com valores tanto verdadeiros (V) quanto falsos (F).

Analisaremos esta questão nas três formas: seja **p** a proposição "Chove" e **~p** sua negação, "Não chove".

"Chove ou não chove" é uma **tautologia**, pois o resultado da tabela-verdade (Tabela 18) é sempre verdadeiro.

Tabela 18 Tabela-verdade da proposição $(p \vee \sim p)$.

p	~p	$(p \vee \sim p)$
V	F	V
F	V	V

Fonte: desenvolvida pelos autores.

"Chove e não chove" é uma **contradição**, pois a tabela-verdade (Tabela 19) possui apenas valores falsos no resultado.

Tabela 19 Tabela-verdade da proposição $(p \wedge \sim p)$.



p	~p	$(p \wedge \sim p)$
V	F	F
F	V	F

Fonte: desenvolvida pelos autores.

"Se chove, então não chove" é uma **contingência**, uma vez que o resultado da tabela (Tabela 20) possui valores verdadeiros e falsos.

Tabela 20 Tabela-verdade da proposição $(p \rightarrow \sim p)$.

p	~p	$(p \rightarrow \sim p)$
V	F	F
F	V	V

Fonte: desenvolvida pelos autores.

Compreendidas as tautologias, as contradições e as contingências, passamos a explorar, no próximo tópico, a elaboração e representação de algoritmos.

3. Elaboração e representação de algoritmos

Os conceitos de lógica apresentados no tópico 1, *Introdução à lógica*, tiveram como objetivo a formalização e a representação dos pensamentos. Por meio da lógica, você pôde perceber que é possível definir, de maneira organizada e coerente, uma sequência de raciocínio e, com isso, atingir os objetivos propostos.

Todo esse ferramental apresentado pode ser empregado na construção de algoritmos e programas.

Assim, quando você for escrever programas para computador, um dos primeiros passos é a criação de algoritmos. Por meio deles, você pode esboçar todos os passos necessários para a resolução de um determinado problema.

Segundo Ascencio (1999), o algoritmo é a descrição de uma sequência de passos que deve ser seguida para a realização de uma tarefa.



Dessa forma, em nosso cotidiano, utilizamos algoritmos a todo instante para realizar atividades, como, por exemplo, ir ao supermercado, trocar o pneu do carro, limpar a casa, passear com o cachorro, ir ao trabalho, dentre outros. A seguir, serão apresentados alguns exemplos de algoritmos do cotidiano, tais como: trocar o pneu do carro e ir ao trabalho.

Analise os passos do algoritmo 3 e, em seguida, responda à questão proposta.

Algoritmo 3 – trocar o pneu do carro

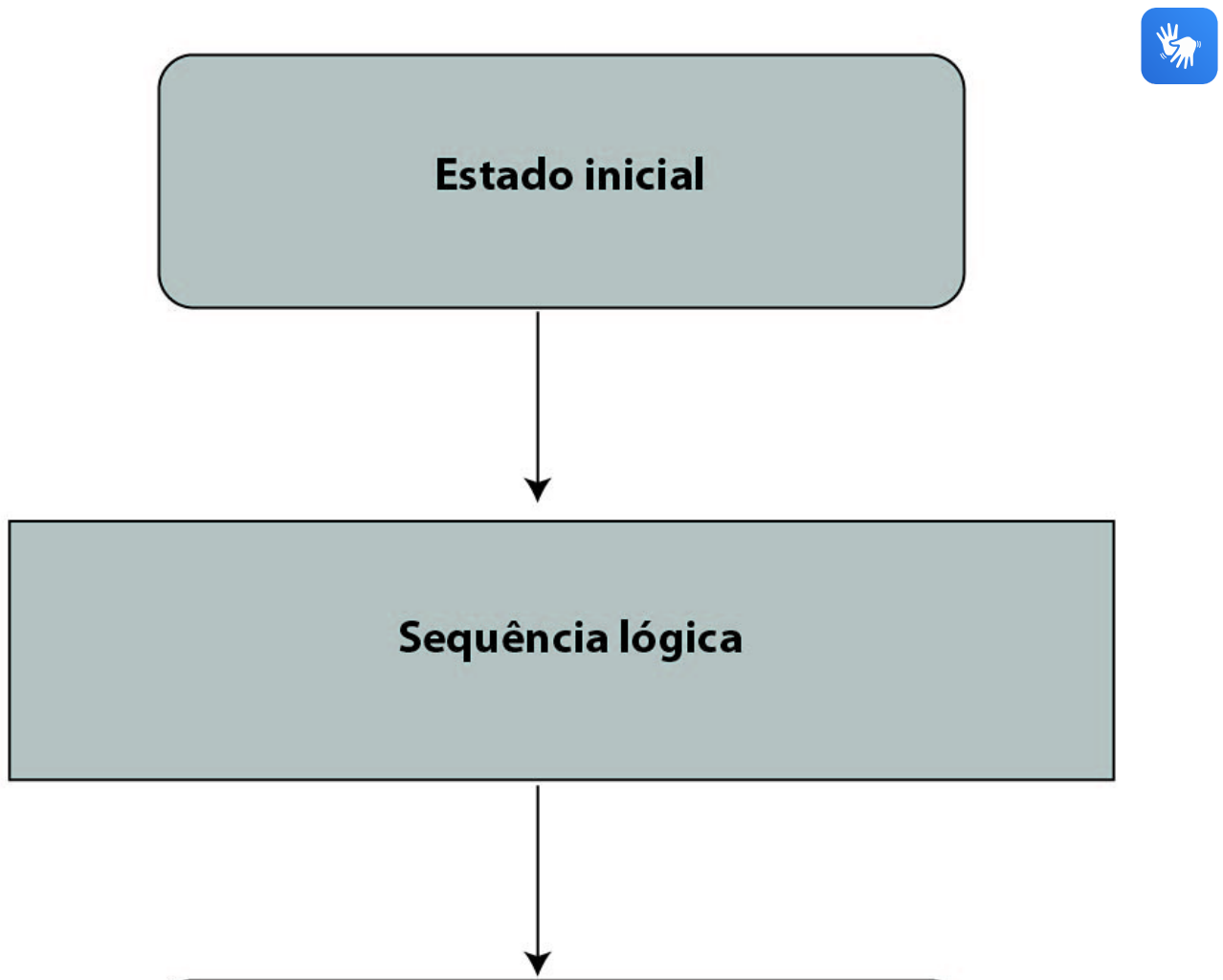
- 1 Desparafusar a roda.
- 2 Levantar o carro com o macaco.
- 3 Retirar a roda com o pneu furado.
- 4 Colocar o estepe.
- 5 Abaixar o carro.
- 6 Parafusar a roda.

Outro exemplo que podemos analisar é o representado pelo algoritmo 4, a seguir:

Algoritmo 4 – ir ao trabalho

- 1 Acordar cedo.
- 2 Levantar da cama.
- 3 Ir ao banheiro.
- 4 Vestir as roupas.
- 5 Tomar café.
- 6 Ir para o trabalho.

A Figura 1, disposta a seguir, ilustra graficamente o fluxo de execução de um algoritmo. O estado inicial representa o ponto de partida do algoritmo, ou seja, o ponto em que a primeira ação será executada. A sequência lógica define o conjunto de passos, desde o início, que devem ser seguidos para atingir os objetivos. É imprescindível que esses passos sejam claros e precisos, caso contrário, o objetivo pode não ser atingido. Por último, o estado final determina qual será o resultado da execução do algoritmo.



Estado final

Fonte: desenvolvida pelos autores.

Figura 1 Fluxo de execução de um algoritmo.

Quando pensamos em programação de computadores, utilizamos os algoritmos para abstrair o raciocínio em um conjunto de passos que devem ser executados pelo computador. Esses passos solucionam um determinado problema.

Podemos, por exemplo, criar um algoritmo computacional para determinar a soma entre dois números, conforme proposto pelo algoritmo a seguir:

Algoritmo 5 – soma entre dois números

- 1 Receber o primeiro número.
- 2 Receber o segundo número.
- 3 Calcular a soma entre o primeiro e o segundo número.
- 4 Exibir o resultado na tela.



Outro exemplo de algoritmo pode ser elaborado para determinar se uma pessoa pode ou não votar. Observe:

Algoritmo 6 – votar

- 1 Receber a idade.
- 2 Se a idade for maior ou igual a 16, exibir "Você já pode votar."
- 3 Se a idade for menor que 16, exibir "Você não pode votar."

Os exemplos de algoritmos apresentados até o momento utilizam a maneira mais simples de representação, que é chamada de descrição narrativa. Além dessa representação, os algoritmos também podem ser representados por meio de fluxogramas e pseudocódigos (ou português) (ASCENCIO, 2002).

Desse modo, podemos dizer que a descrição narrativa é a forma mais simples de elaboração de algoritmos. Esse método tem como objetivo a abstração de problemas em algoritmos por meio da utilização da linguagem natural.

Como já mencionamos, possuímos outra maneira de construir algoritmos, que é por meio de fluxogramas. Nesse caso, a abstração de problemas é representada por meio de símbolos gráficos bem definidos, os quais esboçam uma ação dentro da sequência lógica do algoritmo. Os principais símbolos são ilustrados no Quadro 1. Observe:

Quadro 1 Principais símbolos utilizados na construção de fluxogramas.

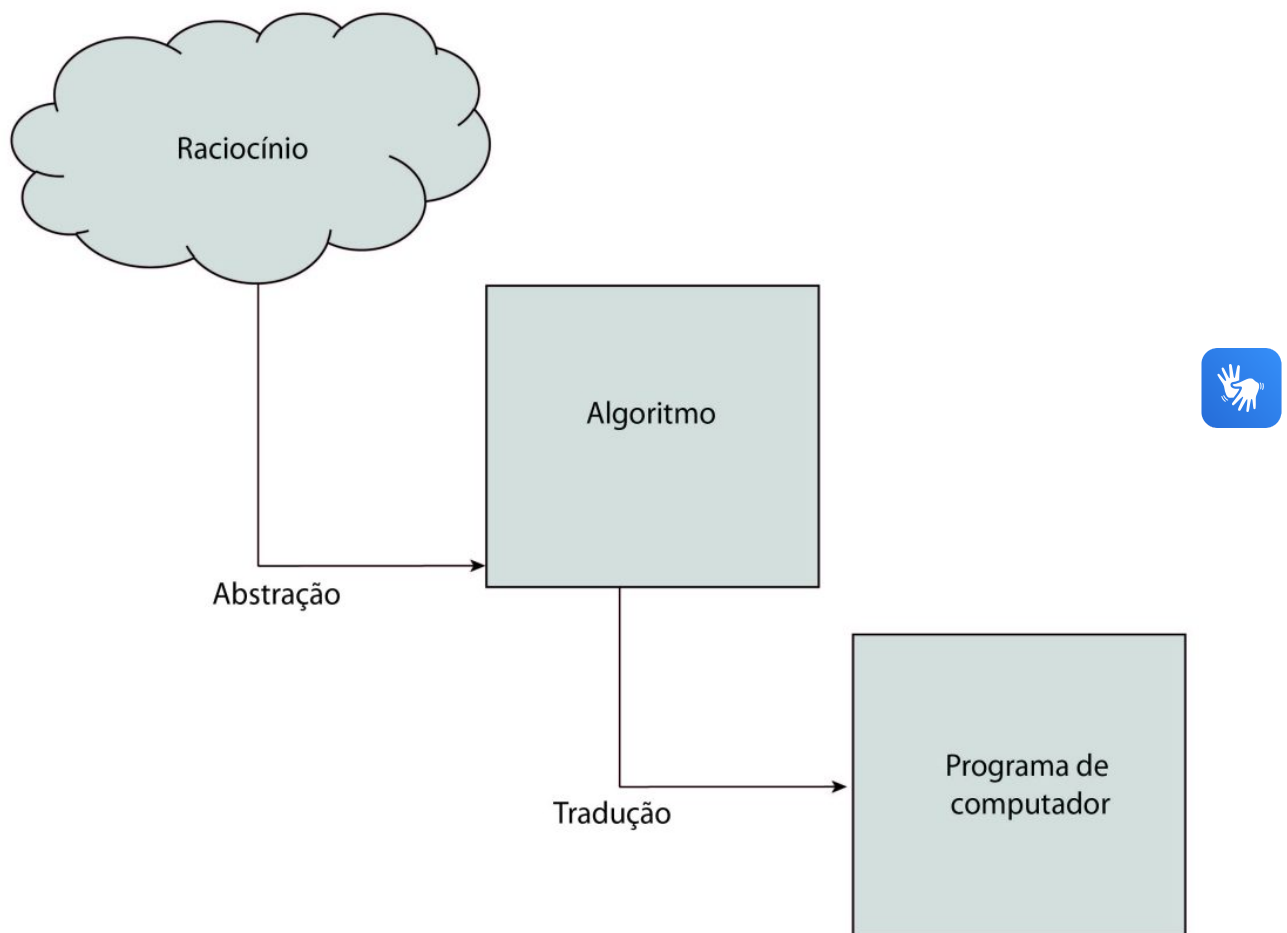
SÍMBOLOS UTILIZADOS NA CONSTRUÇÃO DE FLUXOGRAMAS	
Símbolo	Significado
	Início ou fim do algoritmo.
	Fluxo de dados durante a execução do algoritmo.
	Cálculos realizados pelo algoritmo.
	Entrada de dados.
	Saída de dados.
	Tomadas de decisão.

Fonte: adaptado de Ascencio (2002).

Além dos símbolos ilustrados no quadro (principais símbolos utilizados na construção de fluxogramas), os algoritmos também podem ser representados

por meio de pseudocódigos, cuja elaboração segue regras bem definidas para a resolução de problemas.

Independentemente da forma de elaboração dos algoritmos (descrição narrativa, fluxograma ou pseudocódigo), todas visam à construção de programas. Assim, um programa de computador pode ser definido como a tradução de um algoritmo para uma linguagem que apenas o computador entende, também conhecida como "linguagem de programação". A Figura 2 ilustra o processo de abstração do raciocínio em algoritmo e a tradução em uma linguagem de programação. Veja:



Fonte: desenvolvida pelos autores.

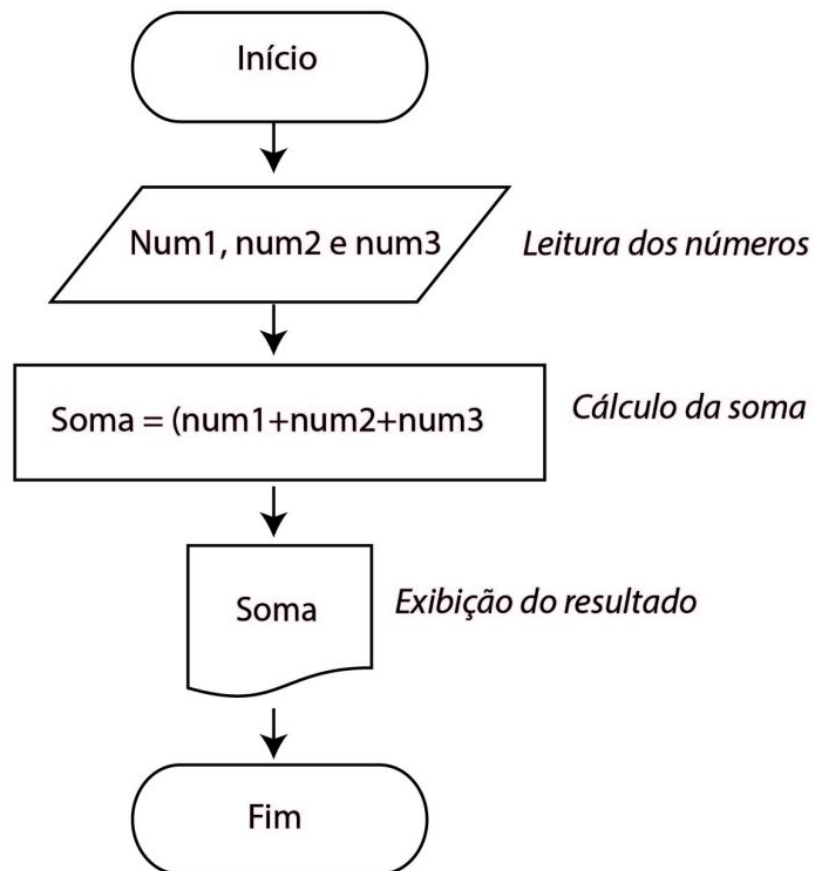
Figura 2 Processo de construção de programas de computador (raciocínio, algoritmo e programa).

Para facilitar o entendimento, vamos, a seguir, elaborar um exemplo completo, utilizando as representações de descrição narrativa, fluxograma e pseudocódigo. Observe que a mesma situação está representada nos três tipos de algoritmos.

Algoritmo 7 – somar três números e apresentar o resultado com descrição narrativa

- 1 Receber o primeiro número.
- 2 Receber o segundo número.
- 3 Receber o terceiro número.
- 4 Somar os três números.
- 5 Apresentar o resultado obtido com a soma.

Agora, visualize o fluxograma (Figura 3) representando esses passos.



Fonte: desenvolvida pelos autores.

Figura 3 Fluxograma da soma de três números.

Por fim, note como deve ser descrita a apresentação do algoritmo pseudocódigo:

```

1      algoritmo "soma três números"
2      //Algoritmo elaborado para somar três números
3      //e apresentar o resultado
4
5      var
6          num1, num2, num3 : inteiro
7          Soma : inteiro
8
9      inicio
10
11         escreva("Entre com o primeiro número:")
12         leia(num1)
13         escreva("Entre com o segundo número:")
14         leia(num2)
15         escreva("Entre com o terceiro número:")
16         leia(num3)
17         Soma <- num1+num2+num3
18         escreva("Resultado = ", soma)
19
20      fimalgoritmo

```

Antes de concluir seus estudos sobre este tópico, responda à seguinte questão:

Para encerrarmos a abordagem dos conteúdos propostos nesta unidade, é prescindível que assista ao vídeo e realize a leitura indicada na sequência.



Será que entendemos efetivamente qual é o conceito de algoritmo?

No vídeo *O que é algoritmo?* (<https://www.youtube.com/watch?v=dvNp575fwzQ>), Rafaella Ballerini esclarece, de forma clara e didática, alguns princípios básicos dos algoritmos. Vale a pena conferir!



Vamos complementar um pouco mais nossos conhecimentos sobre esse assunto?

Para aprofundar seus estudos a respeito dos fundamentos de lógica e da elaboração e representação de algoritmos, recomendamos que faça a leitura complementar do Capítulo 1 – *Introdução à lógica* (p. 10) da obra *Linguagem e lógica de programação* (ALVES, 2014). Esta obra está na dis-

4. Considerações

Chegamos ao final do estudo desta primeira unidade de **Algoritmos e Programação**. Nela, você teve a oportunidade de aprender os principais conceitos a respeito da lógica, como por exemplo, as proposições, os conectivos e as tabelas-verdade. Além disso, pôde estudar os conceitos fundamentais dos algoritmos e conhecer os três tipos de representação: a descrição narrativa, o fluxograma e o pseudocódigo.

Na próxima unidade, estudaremos as variáveis, os tipos de dados e expressões, compreendendo como é a elaboração dos programas baseada em comandos de entrada e saída de dados.

Até mais!



Unidade 2 – Variáveis, Tipos de Dados e Expressões

Objetivos

- Entender como os algoritmos são construídos.
- Elaborar algoritmos utilizando as estruturas sequenciais.

Conteúdos

- Estrutura sequencial.
- **Variáveis** e constantes.
- Expressões aritméticas.
- Expressões lógicas.
- Fundamentos da lógica.
- Elaboração e representação de algoritmos.



Problematização

Como é a estrutura básica de um algoritmo? O que são tipos de dados, variáveis e constantes? Como representá-las em um algoritmo? Como realizar operações lógicas e aritméticas?

Orientações para o estudo

Durante o estudo desta unidade, além dos conteúdos apresentados, você deverá ler os capítulos indicados que se correlacionam com o tema, bem como assistir ao vídeo para compreender, de forma prática, como criar uma estrutura sequencial de algoritmos. Também é fundamental responder às questões para identificar se compreendeu efetivamente os conteúdos abordados e, assim, obter o máximo proveito desta disciplina.

1. Introdução

Na unidade anterior, além de estudar os principais conceitos da lógica matemática, você também teve a oportunidade de conhecer as formas de representação de algoritmos.

Nesta unidade, estudaremos os conceitos essenciais para a construção de algoritmos com a utilização do método do pseudocódigo. Para tanto, inicialmente, você terá a possibilidade de aprender como declarar variáveis e constantes, como atribuir valores a variáveis, como exibir informações na tela e como receber dados digitados pelo usuário.

Em seguida, terá a possibilidade de aprender como utilizar expressões na elaboração de algoritmos e escrever algoritmos que utilizam expressões aritméticas e seus operadores matemáticos. Além disso, você terá condições de discutir as expressões lógicas e seus respectivos operadores lógicos e relacionais.

Bons estudos!

2. Estrutura sequencial



Como você pôde aprender na unidade anterior, a construção de um algoritmo depende da organização lógica e coerente de um conjunto de passos. Esses passos devem ser executados de maneira sequencial e, no caso dos algoritmos, sempre do início para o fim. Além disso, para cada passo ou linha do algoritmo, a execução ocorre sempre da esquerda para a direita.

Antes de dar continuidade aos seus estudos, assista ao vídeo a seguir para entender melhor o que é um algoritmo.

Para desenvolver os algoritmos a partir desta unidade, utilizaremos a plataforma VisuAlg. Por isso, é essencial que assista ao vídeo a seguir, por meio do qual será possível aprender a baixar e configurar essa ferramenta.

Estrutura básica de um algoritmo

Os algoritmos computacionais possuem uma estrutura básica na qual são definidos, por exemplo, o nome do algoritmo, as variáveis utilizadas e o bloco

principal.

A estrutura mínima exigida para um algoritmo é a apresentada no Código 1, conforme descrito a seguir.

Código 1

```
1      algoritmo "nome do algoritmo"
2      //informações sobre o algoritmo
3
4      // declaração das variáveis
5
6      inicio
7
8      {bloco de instruções}
9
10     finalgoritmo
```

Como é possível notar, na **Linha 1**, é definido o nome do algoritmo, uma informação essencial para sua identificação. Esse nome é sempre definido entre "aspas duplas" e aparece após a palavra algoritmo.



Na **Linha 2**, podemos observar um recurso muito utilizado pelos programadores para a documentação dos algoritmos, chamado de "comentário". Os comentários são utilizados apenas para melhorar a legibilidade e facilitar o entendimento do algoritmo. Mais adiante, nos próximos tópicos, serão apresentadas mais informações sobre os comentários.

Já a **Linha 4** determina a parte do algoritmo em que são declaradas as variáveis e as constantes. Esse conceito é fundamental para a construção de algoritmos e também será abordado em breve.

A **Linha 6** contém a palavra reservada início, que determina o início do bloco principal do algoritmo. A partir desse ponto, o programador pode inserir toda a lógica de programação ou bloco de instruções.

Finalmente, na **Linha 10**, é especificado o fim do algoritmo pela palavra reservada finalgoritmo.

Um ponto importante a ser destacado é a **utilização de linhas em branco dentro do algoritmo**, fato que pode ser observado nas Linhas 3, 5, 7 e 9 do Código 1. É um recurso muito utilizado, pois facilita a leitura do algoritmo.

Outro ponto a destacar é a **utilização de tabulações ao longo do algoritmo**, como pode ser notado na Linha 8. As tabulações são imprescindíveis para a organização do algoritmo: da mesma maneira que as linhas em branco, tornam a leitura mais fácil.

Os comentários, as linhas em branco e as tabulações fazem parte das boas práticas de programação.

Para entender, de forma prática, como é a estrutura básica de um algoritmo, bem como a criação de variáveis e a manipulação de valores em memória, assista ao vídeo indicado a seguir.

Atenção!

A aplicação do VisuAlg nesta disciplina se faz, exclusivamente, pela necessidade de apresentar a lógica da forma mais simples possível. O objetivo, neste momento, é trazer luz ao entendimento, especialmente aos alunos que estão iniciando seus estudos na área de algoritmos e programação.



Entendida a estrutura do algoritmo, vamos, na sequência, conhecer quais são os tipos de dados que podem compor um algoritmo.

Tipos de dados

Um algoritmo que não manipula informações pode não ter nenhuma utilidade. A grande essência da construção de algoritmos e programas é a possibilidade de manipulação de grandes volumes de informação.

Essas informações que são manipuladas pelos algoritmos são chamadas de tipos abstratos de dados.

Os tipos abstratos de dados, ou, simplesmente, **tipos de dados**, definem quais tipos de informações podem ser processados pelo computador. Todo tipo abstrato de dados é um conjunto de valores específicos e, também, um conjunto

de operações.

Vamos entender melhor?

O **tipo** abstrato de dados **inteiro** permite a manipulação de **valores numéricos que não possuem a parte decimal**. São exemplos de números inteiros: 10, 8, -23, 183, 3245, 0, -12345. Dentre as operações permitidas por esses números, podemos destacar: a soma, a subtração, a divisão e a multiplicação.

Para manipular **valores numéricos com parte decimal**, é utilizado o **tipo** abstrato de dados **real**. Alguns exemplos de números reais são: 10,32; 943,03; -32,25; 0,43; 41234,98.

As operações realizadas com números inteiros também podem ser empregadas nos números reais. Além disso, outras operações são possíveis, como, por exemplo, o cálculo da raiz quadrada de um número.

Os tipos abstratos de dados também permitem a **manipulação de informações textuais**. Esse tipo é conhecido como **caractere ou literal**. Com o tipo caractere, é possível manipular números (0 [...] 9), letras (A [...] Z, a [...] z) e, também, caracteres especiais (por exemplo, #, @, !, \$, %, *). Entretanto, é importante observar que todos os dados do tipo caractere são representados entre "aspas". Isso permite a distinção entre um dado inteiro 12345 e um dado caractere "12345".

Por fim, o **tipo** de dados abstrato **lógico** permite a manipulação de informações que pode assumir apenas dois valores: **verdadeiro** ou **falso**.

Observe, no Quadro 1, um resumo dos tipos abstratos de dados que podem ser utilizados na manipulação de informações nos algoritmos. São descritas também as suas principais utilizações, bem como alguns exemplos:

Quadro 1 Tipos abstratos de dados.

TIPOS ABSTRATOS DE DADOS

Tipo	Utilização	Exemplo
Inteiro	Números sem casas decimais.	-50 0 132
Real	Números com casas decimais.	-5,42 0,93 866,92
Caractere	Conjunto de caracteres.	"Olá Mundo!" "Hoje é 05 de dezembro de 2015" "Meu e-mail é joao@info.com" (mailto:joao@info.com)
Lógico	Valores verdadeiro ou falso.	Verdadeiro Falso

Fonte: desenvolvido pelos autores.

Constantes, variáveis e comandos de atribuição



Como vimos no subtópico *Tipos de dados*, os algoritmos são capazes de manipular informações com tipos de dados bem definidos.

Dessa forma, quando construímos algoritmos, podemos definir que uma determinada informação não poderá ser alterada ao longo da execução. Esse conceito é chamado de constante. Assim, uma constante é um dado definido no algoritmo que não pode ter seu valor modificado.

Também é possível definir dados que podem ter seu valor alterado durante a execução do algoritmo. Esses dados recebem o nome de variáveis. Quando você declara uma variável em um programa, significa que você está reservando um espaço da memória principal (RAM) para o armazenamento de uma determinada informação. O tamanho do espaço reservado depende do tipo de dados utilizado na variável.

Independentemente da especificação utilizada para a informação, é imprescindível que um tipo abstrato de dado esteja associado a ela. Assim, toda cons-

tante ou variável deve possuir um tipo específico.

Dentro da estrutura básica de um algoritmo, a especificação das variáveis e constantes sempre deve ser definida no cabeçalho. Assim, as variáveis e constantes são declaradas após o nome do algoritmo e antes do bloco principal (início/fimalgoritmo).

Para determinar o início das declarações das variáveis e constantes, é utilizada a palavra reservada declare, como pode ser observado no Código 2:

Código 2

```
algoritmo "variaveis"
// algoritmo para demonstrar
//a declaração de variáveis
var
    nome: caractere
    idade: inteiro
inicio
    nome <- "João da Silva"
    idade <- 21
    escreva(nome, " tem ", idade , " anos")
fimalgoritmo
```



Este algoritmo terá o seguinte resultado da execução: **João da Silva tem 21 anos.**

O Quadro 2 detalhará as linhas de código desse algoritmo:

Quadro 2 Algoritmo inicial.

Sobre	Código
Criação do algoritmo: é inserido o termo algoritmo seguido de seu nome entre aspas "".	algoritmo "variaveis"
Comentários: o código iniciado com // representará uma linha de comentário que não terá efeito na execução do algoritmo.	// algoritmo para demonstrar //a declaração de variáveis

Área de declaração de variáveis: o VisuAlg utiliza a palavra var para determinar o início da declaração das variáveis.	var
Criação de variáveis: deve ser seguido o padrão, iniciando pelo nome da variável, seguido de dois pontos ":" e depois a descrição do tipo da variável.	nome: caractere idade: inteiro
Declaração do início do algoritmo.	Início
Atribuição de valores às variáveis: respeita-se sempre o tipo da variável.	nome <- "João da Silva" idade <- 21
Escrevendo o resultado na tela: são apresentadas as variáveis seguidas de textos, separados por vírgulas. Neste caso, o resultado irá concatenar o valor da variável na memória, seguido do texto comum, compondo um único resultado.	escreva(nome, " tem ", idade , " anos")
Declaração da finalização do algoritmo.	finalgoritmo



Fonte: desenvolvido pelos autores.

Agora, verifique se conseguiu compreender como um algoritmo é executado. Para isso, responda às questões a seguir.

Variáveis e identificadores

O nome de uma variável é denominado identificador. O identificador possui regras muito bem definidas para a especificação de nomes. Além dos nomes das variáveis, os nomes dos algoritmos, das constantes e das sub-rotinas também são considerados identificadores.

Para definir um identificador, algumas regras devem ser seguidas, tais como:

1. Todo identificador deve começar por um caractere do alfabeto.
2. Não é permitido definir identificadores que comecem com números.
3. Não é permitido definir identificadores que comecem com caracteres es-

peciais.

4. Não é permitido utilizar espaços em branco na definição do identificador.

Observe, no Quadro 3, os identificadores válidos e os inválidos:

Quadro 3 Identificadores válidos e inválidos.

IDENTIFICADORES	
Válidos	Inválidos
Nome	lvalor
media_final	#cidade
ENDERECO	nome aluno
X	dia/mes
valor1	"nome"

Fonte: desenvolvida pelos autores.



Alguns identificadores podem ser utilizados apenas com um sentido especial dentro da elaboração do algoritmo. Esses identificadores especiais são denominados **palavras reservadas**.

É importante salientar que palavras reservadas não podem ser utilizadas como identificadores. São exemplos de palavras reservadas: algoritmo, var, início, fimalgoritmo, inteiro, literal, caractere, lógico, dentre outras.

Comandos de entrada e saída

A elaboração de programas não teria sentido se não houvesse a possibilidade de leitura de informações dos usuários. Para que um programa ou algoritmo seja capaz de receber dados digitados pelo usuário, são utilizados os **comandos de entrada**.

Desse modo, quando utilizamos um comando de entrada em um algoritmo, o valor informado pelo usuário é armazenado em uma variável. Em linguagem

algorítmica, o comando de entrada é representado pela palavra **leia**.

Por exemplo, ao inserir a instrução **leia(nome)**, o valor digitado pelo usuário será armazenado na variável "nome".

Outra característica essencial na elaboração de um algoritmo é a capacidade de exibir informações para o usuário na tela ou na impressora. Para isso, são utilizados comandos específicos, que são chamados de comandos de saída.

Um comando de saída tem como objetivo a apresentação de uma informação ou de um conteúdo de uma variável para o usuário. Esse comando é definido pela palavra **escreva**.

Por exemplo, ao inserir a instrução **escreva("Seja bem-vindo!")**, será exibida na tela do usuário a mensagem definida entre aspas.

Assista ao vídeo a seguir para entender melhor.



Para exemplificar a utilização dos comandos de entrada e saída, vamos elaborar um algoritmo que recebe como entrada o valor do salário de um funcionário e calcula um novo salário com reajuste de 25%. O resultado é o apresentado na tela:

Código 3

```

algoritmo "comandos_entrada_saida"
// algoritmo para demonstrar a utilização
// dos comandos de entrada e saída
//declaração das variáveis
var
    salario_atual: real
    salario_novo: real

inicio

    //leitura das informações
    escreva("Informe o salário: ")
    leia(salario_atual)

    //cálculo do novo salário}
    salario_novo <- salario_atual + (salario_atua * (25/100))

    //escrita do resultado na tela
    escreva("Salário com aumento = R$ ", salario_novo)

fimalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução a seguir. As informações utilizadas como entrada são apenas ilustrativas.



Resultado da execução do algoritmo

```

Informe o salário: 1000,00
Salário com aumento = R$ 1250,00

```

No Código 4, é demonstrada a utilização do comando de saída na exibição de diversas variáveis ao mesmo tempo. O algoritmo elaborado tem como objetivo o cálculo do Índice de Massa Corpórea (IMC) de uma pessoa. Observe:

Código 4

```

algoritmo "calcula_imc"
//algoritmo que demonstra como calcular
//o índice de massa corpórea (IMC)
//declaração das variáveis
var
    nome: caractere
    peso, altura, imc: real

inicio
    //leitura das informações
    escreva("Nome: ")
    leia(nome)
    escreva("Peso: ")
    leia(peso)
    escreva("Altura: ")
    leia(altura)

    //cálculo do índice de massa corpórea
    imc <- peso / (altura*altura)

    //escrita do resultado na tela
    escreva(nome, " seu IMC é igual a ", imc:5:2)

fimalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução a seguir:



Resultado da execução do algoritmo:

```

Nome: João da Silva Peso: 72,5
Altura: 1,82
João da Silva seu IMC é igual a 21,88

```

Você conseguiu compreender a execução dos comandos de entrada e saída a partir desses exemplos? Para ter certeza de que aprendeu os comandos, responda à questão a seguir:

Comentários

Os comentários são um recurso imprescindível para uma boa prática de programação. Por meio deles, o programador pode documentar o algoritmo e, com isso, aumentar a clareza do código.

Um aspecto importante a respeito dos comentários é que eles são desconsiderados durante a execução do algoritmo ou do programa. Isso significa que eles não interferem no conteúdo do algoritmo. Por isso, podemos escrever qualquer tipo de informação para facilitar o seu entendimento.

Você pode fazer dois tipos de comentários no seu algoritmo: comentários de única linha ou de múltiplas linhas.

Os comentários de uma única linha são definidos pelos caracteres `//`, como, por exemplo:

```
1 //este é um comentário de uma única linha
```

E os comentários de múltiplas linhas são aqueles nos quais podemos escrever o quanto for necessário entre o bloco de chaves `{ ... }`, conforme é possível observar no exemplo a seguir:

```
1 {No comentário de múltiplas linhas,  
2 podemos explicar, detalhadamente, uma  
3 determinada parte do algoritmo}
```



Após compreender a estrutura sequencial de criação dos algoritmos, vamos conhecer os operadores aritméticos, expressões matemáticas também relevantes na construção de algoritmos.

3. Expressões aritméticas

Você sabia que uma das grandes habilidades dos programas de computadores é a capacidade de realizar cálculos matemáticos? Para isso, os programas utilizam um conjunto de símbolos especiais, que são chamados de **operadores aritméticos**.

Para realizar a soma entre dois números, como, por exemplo, **10** e **20**, é necessária a utilização do operador aritmético de adição. Esse operador é representado pelo símbolo **+**. Assim, a instrução **soma = 10+20**, realiza a soma entre os valores e atribui o resultado à variável **soma**.

Antes de dar continuidade, assista ao vídeo a seguir para entender melhor os operadores lógicos e aritméticos.

As principais operações matemáticas possuem operadores aritméticos, os quais podem ser aplicados a qualquer tipo de expressão contendo números inteiros ou números reais. Observe, no Quadro 4, os principais operadores aritméticos.

Quadro 4 Operadores aritméticos.

OPERADORES ARITMÉTICOS		
Operação	Operador	Exemplo
Adição	+	$10 + 20$
Subtração	-	$9 - 2$
Multiplicação	*	$2 * 4$
Divisão	/	$6 / 3$

Fonte: desenvolvido pelos autores.



Os parênteses podem ser utilizados na construção de expressões algorítmicas da mesma maneira que são empregados nas expressões matemáticas. Por exemplo:

```
1      resultado <- (10 + 20) * ( 5 - ( 9 / 3 ) )
```

Além disso, é possível utilizar variáveis na construção de expressões. Observe:

```
1      a <- 10
2      b <- 5
3      c <- 5
4      d <- 9
5      e <- 3
6      resultado <- (a + b) * ( c - (d / e) )
```

Para auxiliar no seu aprendizado, assista ao vídeo indicado a seguir. Nele, são abordados os seguintes temas iniciais do desenvolvimento de algoritmos:

- Comentários ilustrativos em algoritmos.
- Criação de variáveis.
- Comandos de entrada e saída de dados.
- Cálculo entre variáveis.
- Exibindo resultados.

Além dos operadores matemáticos básicos que estudamos até aqui, vamos conhecer, a seguir, os operadores de potenciação e radiação.

No conjunto de operadores matemáticos, também é possível representar as operações de **potenciação** e **radiciação**. Para isso, utilizamos os operadores **pot** e **raiz**.

Como sabemos, a potenciação realiza a operação de um número elevado a outro, como, por exemplo, 23 (2 elevado a 3). No algoritmo, representamos essa operação como **2^3**.

Atenção!

No VisuAlg, a operação de potenciação é representada pelo símbolo $\wedge$. Assim, a instrução do código de cálculo deve ser expressa da seguinte maneira: **resultado <- A^B**.



O algoritmo do Código 5, descrito a seguir, ilustra a funcionalidade do operador matemático de potenciação. Observe:

Código 5

```

algoritmo "potenciacao"
//algoritmo para demonstrar o operador matemático
//de potenciação
var
    A, B: inteiro
    resultado: real
inicio
    //leitura das informações
    escreva("Informe o valor de A: ")
    leia(A)
    escreva("Informe o valor de B: ")
    leia(B)

    cálculo de A elevado a B}
    resultado <- A^B

    //resultado
    escreva(A, " elevado a ", B, " é igual a ", resultado)

fimalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução a seguir. As informações utilizadas como entrada são apenas ilustrativas.



Resultado da execução do algoritmo

```

Informe o valor de A: 2
Informe o valor de B: 3
2 elevado a 3 é igual a 8

```

Já a operação de radiciação, responsável pelo cálculo da raiz quadrada de um número, é representada pelo operador **raizq**. Assim, a raiz quadrada de 25 é formalizada pela operação **raizq(25)**.

No próximo exemplo, é demonstrada a utilização do operador de radiciação. Assim, diante de um número informado pelo usuário, é realizado o cálculo da raiz quadrada. Veja:

Código 6


```

algoritmo "raiz_quadrada"
  //algoritmo para demonstrar o operador
  //matemático de radiciação
var
  A: real
  resultado: real
inicio
  //leitura das informações
  escreva("Informe o valor de A: ")
  leia(A)
  //cálculo da raiz quadrada A
  resultado <- raizq(A)
  //resultado
  escreva("Raiz quadrada de ", A, " é igual a ", resultado)
finalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```

Informe o valor de A: 25
Raiz quadrada de 25 é igual a 5

```



Compreendida a estrutura de um algoritmo, resolva os exercícios propostos a seguir para aplicar os conceitos aprendidos na construção dos algoritmos.

4. Exercícios resolvidos

Neste tópico, você encontrará alguns exercícios resolvidos e comentados que facilitarão a sua compreensão e o auxiliarão em sua aprendizagem.

Exercício 1

Construa um algoritmo que calcule a área de um triângulo. Lembre-se de que, para realizar o cálculo da área do triângulo, é necessário utilizar a fórmula:

$$area = \frac{base * altura}{2}$$

Código 7

```
algoritmo "e_areatriangulo"
//algoritmo para calcular a área de um triângulo
var
    area, base, altura: real
inicio
    escreval("Cálculo da Área do Triângulo")
    escreva("Informe a base: ")
    leia(base)
    escreva("Informe a altura: ")
    leia(altura)
    //cálculo da área do triângulo
    area <- (base * altura) / 2
    //resultado
    escreva("A área do triângulo é = ", area:5:2)
finalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:



```
Cálculo da Área do Triângulo Informe a base: 6,5
Informe a altura: 2,2
A área do triângulo é = 7,15
```

Exercício 2

Escreva um algoritmo que receba um número inteiro do teclado e imprima na tela, tanto o seu sucessor (numero+1) quanto o seu antecessor (numero -1):

Código 8

```
algoritmo "e_sucessor_antecessor"
// algoritmo para calcular o sucessor e
//antecessor de um número inteiro
var
    numero: inteiro
inicio
    //leitura das informações
    escreva("Informe um número: ")
    leia(numero)
    //escrita do resultado na tela
    escreval("Sucessor = ", (numero+1))
    escreva("Antecessor = ", (numero-1))
finalgoritmo
```

Resultado da execução do algoritmo:

```
Informe um número: 10
Sucessor = 11
Antecessor = 9
```

Exercício 3

Elabore um algoritmo que realize a conversão de moedas. O algoritmo deve solicitar ao usuário a cotação do dólar, bem como a cotação do euro. Em seguida, o usuário informará o valor, em reais, que deseja converter. Após os cálculos necessários, o algoritmo deverá exibir na tela:

1. O valor em reais convertido para dólar.
2. O valor em reais convertido para euro.

Código 9



```

algoritmo "e_conversor_de_moedas"
//algoritmo para converter moedas }
var
    cotacao_dolar: real
    cotacao_euro: real
    valor_reais, valor_dolar, valor_euro: real
inicio
    escreva("Informe um valor para conversão (em reais): ")
    leia(valor_reais)
    escreva("Cotação do Dólar: ")
    leia(cotacao_dolar)
    escreva("Cotação Euro: ")
    leia(cotacao_euro)
    //conversão real para dólar
    valor_dolar <- valor_reais / cotacao_dolar
    //conversão real para euro
    valor_euro <- valor_reais / cotacao_euro
    //resultado
    escreval("Valor em dólar = ", valor_dolar:8:2)
    escreva("Valor em euro = ", valor_euro:8:2)
fimalgoritmo

```

Resultado da execução do algoritmo:

```

Informe um valor para conversão (em reais): 100,00
Cotação do Dólar: 5,44
Cotação Euro: 6,14
Valor em dólar = 18,38
Valor em euro = 16,29

```



Exercício 4

Elabore um algoritmo que seja capaz de ler o nome e o sobrenome de uma pessoa e armazenar essas informações em duas variáveis do tipo caractere. Por exemplo, a variável nome recebe o valor **João**, e a variável sobrenome recebe o valor **da Silva**. O algoritmo deverá apresentar na tela essas informações concatenadas, da seguinte maneira: **João da Silva**.

Código 10

```

algoritmo "concatenarinformacoes"
//Exemplo de algoritmo que demonstra a concatenação
//de duas variáveis contendo um conjunto de caracteres.
var
    nome: caractere
    sobrenome: caractere
    nomecompleto: caractere
inicio
    escreva("Informe seu nome:")
    leia(nome)
    escreva("Informe seu sobrenome:")
    leia(sobrenome)
    //concatenação das variáveis
    nomecompleto <- nome + " " + sobrenome
    escreva("O seu nome completo é: ", nomecompleto)
finalgoritmo

```

Resultado da execução do algoritmo:

```

Informe seu nome: João
Informe seu sobrenome: da Silva
O seu nome completo é: João da Silva

```



Exercício 5

Elabore um algoritmo capaz de efetuar a troca de valores entre duas variáveis. Por exemplo: seja A uma variável do tipo inteiro contendo o valor 10 e B outra variável do tipo inteiro contendo o valor 20; o algoritmo deverá ser capaz de manipular essas informações de forma que, ao final, a variável A tenha o valor 20 e a variável B tenha o valor 10. Depois de elaborado o algoritmo, apresente os resultados na tela. Os valores das variáveis A e B devem ser informados pelo usuário.

Código 11

```

algoritmo "troca_entre_variaveis"
//Algoritmo que demonstra a troca de valores
//entre duas variáveis.
var
    valor1: inteiro
    valor2: inteiro
    auxiliar: inteiro
inicio
    escreval("TROCA DE VALORES ENTRE DUAS VARIÁVEIS")
    escreval("")
    escreva("Informe o primeiro valor: ")
    leia(valor1)
    escreva("Informe o segundo valor: ")
    leia(valor2)
    //exibição dos valores originais das variáveis
    escreval("")
    escreval("VALORES ORIGINAIS")
    escreval("Valor 1: ", valor1)
    escreval("Valor 2: ", valor2)
    //troca de valores entre as variáveis
    auxiliar <- valor1
    valor1 <- valor2
    valor2 <- auxiliar
    //saída de dados
    escreval("")
    escreval("VALORES APÓS A TROCA")
    escreval("Valor 1: ", valor1)
    escreval("Valor 2: ", valor2)
fimalgoritmo

```



Resultado da execução do algoritmo:

```

TROCA DE VALORES ENTRE DUAS VARIÁVEIS
Informe o primeiro valor: 10
Informe o segundo valor: 20

VALORES ORIGINAIS
Valor 1: 10
Valor 2: 20

VALORES APÓS A TROCA
Valor 1: 20
Valor 2: 10

```

Exercício 6

Os juros representam a remuneração de um capital empregado em alguma ati-

vidade produtiva. Nesse sentido, um dos cálculos empregados na área financeira é a determinação dos juros simples. Esse tipo de juro utiliza como base o número de períodos que o capital foi aplicado, a taxa unitária do período e o valor principal que foi empregado. Com base nessas informações, o cálculo é realizado por meio da fórmula: $J = P * i * n$, em que **J** representa o resultado do cálculo dos juros simples; **P** o valor principal; **i** a taxa unitária ao período; e **n** o número de períodos. De acordo com as informações, elabore um algoritmo capaz de calcular os juros simples. Para isso, o algoritmo deverá solicitar as informações ao usuário e, ao final, apresentar na tela o cálculo dos juros:

Código 12

```
algoritmo "calculojuros_simples"
//algoritmo que demonstra o cálculo de juros simples
var
    valor_principal: real
    numero_periodos: real
    taxa_unitaria: real
    juros_simples: real
inicio
    //entrada de dado
    escreval("JUROS SIMPLES")
    escreval("") //pular uma linha
    escreva("Informe o valor principal: ")
    leia(valor_principal)
    escreva("Informe o número de períodos: ")
    leia(numero_periodos)
    escreva("Informe a taxa unitária (Exemplo, para 15% informe 0.15): ")
    leia(taxa_unitaria)
    //cálculo dos juros simples
    //Fórmula: J = Pin
    juros_simples <- valor_principal * taxa_unitaria * numero_periodos
    //saída de dados
    escreval("")
    escreval("RESULTADOS")
    escreval("Valor principal: R$ ", valor_principal:4:2)
    escreval("Taxa unitária: ", (taxa_unitaria*100), "%")
    escreval("Número de períodos: ", numero_periodos)
    escreval("JUROS SIMPLES: R$ ", juros_simples:4:2)
Fimalgoritmo
```



Resultado da execução do algoritmo:

JUROS SIMPLES

Informe o valor principal: 1000.00 Informe o número de períodos: 12
Informe a taxa unitária (Exemplo, para 15% informe 0.15): 0.25

RESULTADOS

Valor principal: R\$ 1000.00 Taxa unitária: 25%
Número de períodos: 12
JUROS SIMPLES: R\$ 3000.00

Após a resolução dos exercícios, vamos aprender um pouco mais?



Que tal ampliar sua compreensão a partir de obras relevantes desta área?

Para fixar ainda mais seus conhecimentos e complementar seus estudos, recomendamos que faça as seguintes leituras:

- Capítulo 4 – *A Pseudolinguagem* (p. 23), da obra *Linguagem e lógica de programação* (ALVEZ, 2014).
- Capítulo 2 – *Algoritmo e a lógica de programação* (p. 29), da obra *Algoritmos – Lógica para Desenvolvimento de Programação de Computadores* (MANZANO; OLIVEIRA, 2019). Estas obras estão disponíveis na Minha Biblioteca.



5. Considerações

Nesta unidade, você teve oportunidade de conhecer os principais conceitos para a construção de algoritmos. Também teve oportunidade de estudar os comandos de entrada e saída de informações, além das expressões lógicas e aritméticas.

Após estudar todos os exercícios resolvidos, terminamos o conteúdo desta unidade. Este é o momento para treinarmos os conceitos apresentados ao longo do seu desenvolvimento.

Na próxima unidade, estudaremos os comandos das estruturas condicionais e estruturas de repetição. Aprenderemos a aplicar *loops* de execução contínua com autossoma de variáveis.

Até lá e bons estudos!



Unidade 3 – Estruturas de Controle

Objetivos

- Controlar o fluxo de execução dos algoritmos por meio das estruturas condicionais.
- Especificar blocos de comandos por meio de estruturas de repetição.

Conteúdos

- Estruturas de controle.
- Estruturas condicionais.
- Estruturas de repetição.

Problematização

Como utilizar as estruturas condicionais simples e compostas? Como utilizar as estruturas de repetição “repita”, “enquanto” e “para”?



Orientações para o estudo

Esta unidade está estruturada para que você compreenda as estruturas de controle. Para tanto, é imprescindível realizar as leituras e assistir ao vídeo indicado no decorrer desta unidade. Além disso, organize seu tempo para implementar no VisuAlg os algoritmos disponíveis no tópico *Exercícios resolvidos*. Aproveite para comparar a saída da sua implementação com a saída de cada algoritmo. Essas implementações serão fundamentais para seu aprendizado. Codifique também todos os exemplos apresentados ao longo da unidade. Eles serão fundamentais para complementar seu aprendizado.

Bons estudos!

1. Introdução

Nesta unidade, estudaremos as duas principais estruturas de controle utilizadas na elaboração de algoritmos e programas, conhecidas como **estruturas condicionais** e **estruturas de repetição**.

Como tivemos a oportunidade de aprender na unidade anterior, a execução de um algoritmo sempre segue um fluxo sequencial, que vai do início para o fim. Assim, com as estruturas de controle, podemos determinar como esse fluxo deverá ser executado.

As estruturas condicionais possibilitam ao programador definir se um determinado bloco do algoritmo deve ou não ser executado. Nesse caso, a execução do bloco dependerá da avaliação de uma expressão lógica.

Já as estruturas de repetição permitem ao programador executar um bloco de comandos por um determinado número de vezes ou, até mesmo, executar esse bloco até que uma determinada condição aconteça.

Antes de iniciarmos a abordagem dos conteúdos, tente responder ao seguinte questionamento.



2. Estrutura condicional

Quando você utiliza uma estrutura condicional na elaboração de um algoritmo, é possível selecionar, com base em uma ou mais condições, qual parte do algoritmo deverá ser executada.

Há dois tipos de estruturas condicionais:

1. Estrutura condicional simples.
2. Estrutura condicional composta.

Essas estruturas são representadas em linguagem algorítmica pela palavra reservada **se**. Na sequência, vamos entender cada uma delas, mas, antes disso, assista ao vídeo a seguir que aborda as estruturas de decisão.

Estrutura condicional simples

Uma **estrutura condicional simples** permite ao programador definir que um bloco do algoritmo será executado se uma determinada condição ocorrer. Para definirmos uma estrutura condicional **se**, utilizamos o seguinte modelo:

```
Se condição entao
  //comandos
fimse
```

Você pode entender a estrutura **se** da seguinte forma:

Se uma determinada condição ocorrer, **então** o bloco de comandos será executado.

O bloco de comandos do algoritmo anterior será executado apenas quando a condição especificada for verdadeira. Lembre-se de que a condição é uma expressão lógica e, por isso, sempre retorna um valor verdadeiro ou falso.



Observe o exemplo a seguir:

```
Se a = 0 entao
  //comandos
fimse
```

Nesse caso, a mensagem “**O valor informado é igual a zero**” será exibida na tela apenas quando o valor da variável A for igual a zero.

Veja, no Quadro 1, alguns possíveis exemplos associados à variável A, bem como o resultado da avaliação da expressão condicional.

Quadro 1 Exemplos de expressões condicionais.

EXEMPLOS DE EXPRESSÕES CONDICIONAIS		
Valor da Variável A	Condição	Resultado

2	$2 = 0$	Falso
-5	$-5 = 0$	Falso
0	$0 = 0$	Verdadeiro

Fonte: desenvolvido pelos autores.

Agora, tente responder a mais uma questão.

O algoritmo presente no Código 1 demonstra a utilização da estrutura condicional simples juntamente com as expressões lógicas. Nesse exemplo, são utilizados os operadores de igualdade e os relacionais. Observe:

Código 1

```

algoritmo "estrutura_condicional_simples"
//algoritmo para demonstrar a utilização da estrutura condicional simples
var
  A,B: real
inicio
  escreva("Informe o valor de A: ")
  leia(A)
  escreva("Informe o valor de B: ")
  leia(B)
  //operadores de igualdade
  se (A = B) então
    escreval("A é igual a B")
  fimse
  se (A <> B) então
    escreval("A é diferente de B")
  fimse
  //operadores relacionais
  se (A > B) então
    escreval("A é maior que B")
  fimse
  se (A < B) então
    escreval("A é menor que B")
  fimse
  se (A >= B) então
    escreval("A é maior ou igual a B")
  fimse
  se (A <= B) então
    escreval("A é menor ou igual a B")
  fimse
Fimalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Informe o valor de A: 10
Informe o valor de B: 5
A é diferente de B
A é maior que B
A é maior ou igual a B

Informe o valor de A: 10
Informe o valor de B: 10
A é igual a B
A é maior ou igual a B
A é menor ou igual a B
```

Compreendida a estrutura condicional simples, vamos conhecer, na sequência, a estrutura condicional composta.

Estrutura condicional composta



Há outra maneira de controlar o fluxo de execução de um algoritmo: por meio de uma **estrutura condicional composta**. Ao contrário das estruturas condicionais simples, que executam um bloco de instruções apenas quando o valor da condição é verdadeiro, as estruturas condicionais compostas podem executar um determinado bloco de comandos quando a condição for verdadeira ou executar outro bloco de comandos quando a condição for falsa.

Uma condicional composta é definida pela seguinte estrutura:

```
Se condição entao
    //comandos
Senao
    //comandos
fimse
```

Observe que, no algoritmo anterior, o bloco de comandos da condição **se** será executado apenas quando o resultado da condição for verdadeiro. Em contrapartida, o bloco de comandos da condição **senao** será executado apenas quan-

do a condição for falsa. Como uma condicional pode assumir apenas o valor verdadeiro ou falso, nunca será possível executar os dois blocos de comandos, ou seja, quando um bloco de comandos for executado, o outro não será. Por exemplo:

```
Se A = 0 entao
    escreva("0 valor informado é igual a zero")
Senao
    escreva("0 valor informado é diferente de zero")
fimse
```

Considerando que a variável A pode assumir valores negativos, positivos ou iguais a zero, as possíveis saídas para esse algoritmo são as apresentadas no Quadro 2. Observe:

Quadro 2 Possíveis saídas para o algoritmo com variável A.

POSSÍVEIS SAÍDAS PARA O ALGORITMO			
Valor da Variável A	Condição	Resultado da condição	Saída
2 (positivo)	$2 = 0$	Falso	Diferente de zero
-5 (negativo)	$-5 = 0$	Falso	Diferente de zero
0 (igual a zero)	$0 = 0$	Verdadeiro	Igual a zero

Fonte: desenvolvido pelos autores.

Agora que introduzimos os conceitos de estrutura condicional simples e composta, é imprescindível que assista ao vídeo indicado a seguir para aprofundar ainda mais seu conhecimento a respeito desse assunto.

Vamos entender de forma prática? O exemplo proposto no subtópico a seguir demonstrará a utilização da estrutura condicional aninhada na resolução de problemas. Acompanhe.

Alguns enunciados resolvidos com algoritmos

Considere um algoritmo que receba informações como o tipo do funcionário

(G = gerente, V = vendedor) e o valor do salário mensal.

Para funcionários do tipo gerente, o usuário deverá informar a cota de vendas da loja. Para as lojas que atingirem um total de vendas superior a R\$ 100.000,00, os gerentes recebem um bônus de R\$ 350,00 no salário; caso contrário, recebem apenas o salário.

Já para funcionários do tipo vendedor, o usuário deverá informar o total vendido. Os vendedores que venderem acima de R\$ 5.000,00 ganham uma comissão de 5% sobre o valor vendido; caso contrário, a comissão é de 2%. Observe:

Código 2

```
algoritmo "estrutura_condicional_simples"
//algoritmo para demonstrar a utilização da estrutura condicional simples
var
    A,B: real
inicio
    escreva("Informe o valor de A: ")
    leia(A)
    escreva("Informe o valor de B: ")
    leia(B)
    //operadores de igualdade
    se (A = B) então
        escreval("A é igual a B")
    fimse
    se (A <> B) então
        escreval("A é diferente de B")
    fimse
    //operadores relacionais
    se (A > B) então
        escreval("A é maior que B")
    fimse
    se (A < B) então
        escreval("A é menor que B")
    fimse
    se (A >= B) então
        escreval("A é maior ou igual a B")
    fimse
    se (A <= B) então
        escreval("A é menor ou igual a B")
    fimse
Fimalgoritmo
```



Atenção!

O algoritmo demonstrado apresenta as principais condições. É imprescindível compreendê-las para conseguir elaborar as estruturas lógicas dos programas.

Aplicando as condições anteriormente descritas, temos o algoritmo a seguir.

Código 3

```
algoritmo "estrutura_condicional_aninhada"
//algoritmo que demonstra a utilização da condicional aninhada
var
    salario, novosalario: real
    cotavendas, totalvendas: real
    tipofuncionario: caractere
inicio
    escreva("Tipo funcionário [g=gerente; v=vendedor] : ")
    leia(tipofuncionario)
    escreva("Salário: ")
    leia(salario)
    se (tipofuncionario = "g") entao
        escreva("Cota de vendas: ")
        leia(cotavendas)
        se (cotavendas >= 100000) entao
            novosalario <- salario + 350.00
            escreval("Salário mensal = ", novosalario)
        senao
            escreval("Salário mensal = ", salario)
        fimse
    senao
        se (tipofuncionario = "v") entao
            escreval("Total vendido: ")
            leia(totalvendas)
            se (totalvendas >= 5000) entao
                //comissão de 5%
                novosalario <- salario + (salario * (5/100))
            senao
                //comissao de 2%
                novosalario <- salario + (salario * (2/100))
            fimse
            escreval("Salário mensal = ", novosalario)
        senao
            escreval("Tipo de funcionário inválido.")
        fimse
    fimse
fimse
fimalgoritmo
```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Tipo funcionário [g=gerente; v=vendedor] : g
Salário: 2500,00
Cota de vendas: 125.000,00
Salário mensal = 2850,00
```

O Código 4, a seguir, ilustra a utilização dos operadores lógicos nas estruturas condicionais. Esse exemplo tem como objetivo receber duas notas, calcular a média aritmética e exibir uma mensagem segundo os dados do Quadro 3.

Quadro 3 Critérios e mensagens exibidas pelos algoritmos.

CRITÉRIOS E MENSAGENS EXIBIDAS PELO ALGORITMO	
Média aritmética	Mensagem
maior ou igual a zero e menor que 2,0	reprovado
maior ou igual a 2,0 e menor que 5,0	exame
maior ou igual a 5,0	aprovado

Fonte: desenvolvido pelos autores.

Código 4

```

algoritmo "media_notas"
var nota1, nota2: real
    media: real

inicio

    {leitura das variáveis}
    escreva("Informe a 1ª nota : ")
    leia(nota1)
    escreva("Informe a 2ª nota : ")
    leia(nota2)

    {cálculo da média}
    media <- (nota1+nota2) /2
    escreva("Média = ", media)

    se ( ( media >= 0 ) e ( media < 2 ) ) entao
        escreva("Reprovado")
    senao
        se ( ( media >= 2 ) e ( media < 5 ) ) entao
            escreval("Exame")
        senao
            escreval("Aprovado")
        fimse
    fimse

fimse

fimalgoritmo

```



Resultado da execução do algoritmo:

```

Informe a 1ª nota : 6,5
Informe a 2ª nota : 4,5
Média = 5,5
Aprovado

```

O próximo exemplo ilustra a utilização do operador lógico ou em estruturas condicionais. Nele, o algoritmo simula o funcionamento do sistema de *login* de um *site*.

Código 5

```

algoritmo "login"
//algoritmo que demonstra a utilização do operador OU
var
    usuario: caractere
    senha: inteiro
inicio
    //leitura das variáveis
    escreva("Usuario: ")
    leia(usuario)
    escreva("Senha: ")
    leia(senha)
    se ((usuario <> "joao") ou (senha <> 123)) entao
        escreva("Usuário ou senha inválida.")
    senao
        escreva("Olá João, seja bem-vindo ao sistema!")
    fimse
finalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```

Usuario: João
Senha: 123
Olá João, seja bem-vindo ao sistema!

Usuario: João
Senha: 4321
Usuário ou senha inválida.

```



Além das estruturas condicionais simples e compostas, que vimos anteriormente, é possível controlar o fluxo de execução de um algoritmo por meio de um recurso conhecido como estrutura de seleção múltipla ou estrutura de seleção de múltipla escolha, assunto que estudaremos a seguir.

Estrutura de seleção múltipla

A estrutura de seleção múltipla permite ao programador testar o valor de uma determinada variável (que pode ser inteiro ou caractere) contra um conjunto de valores. Caso o valor da variável seja igual a um valor presente no conjunto, o bloco de comandos associado ao valor do conjunto é executado. Essa estru-

tura é definida pela sintaxe a seguir:

```
escolha(expressao)
  caso condicao
    //bloco de comandos
  caso condicao
    //bloco de comandos
  outrocaso
    //bloco de comandos
fimescolha
```

Uma das opções (ou casos) da estrutura de seleção de múltipla escolha é definida como **outrocaso**.

Essa opção é executada quando nenhuma opção é escolhida pela expressão.

O Código 6 ilustra o funcionamento da estrutura de seleção múltipla. Nesse exemplo, o usuário informará o código do tipo do funcionário e, com base nesse código, deverá ser exibido na tela o cargo do funcionário de acordo com as informações do Quadro 4.



Quadro 4 Códigos dos funcionários.

CÓDIGOS DOS FUNCIONÁRIOS	
Códigos	Cargo
1	Diretor
2	Gerente de projeto
3	Programador
4	Estagiário

Fonte: desenvolvido pelos autores.

Código 6

```

Algoritmo "empr"
var cod : inteiro
inicio
    escreva("Informe o tipo do funcionário: ")
    leia(cod)
    se (cod = 1) entao
        escreval("Diretor.")
    fimse
    se (cod = 2) entao
        escreval("Gerente de projeto.")
    fimse
    se (cod = 3) entao
        escreval("Programador.")
    fimse
    se (cod = 4) entao
        escreval("Estagiário.")
    fimse
finalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:



```

Informe o tipo do funcionário: 3
Programador

```

No próximo exemplo, utilizaremos a estrutura de seleção múltipla para determinar o preço final de um produto. Para tanto, o algoritmo deve receber o total da compra e o código do tipo de pagamento e, em seguida, calcular o preço final do produto em função do tipo de pagamento. Observe que, nesse exemplo, serão utilizadas uma estrutura condicional e uma estrutura de seleção múltipla.

Quadro 5 Parâmetros de entrada do algoritmo.

PARÂMETROS DE ENTRADA DO ALGORITMO			
Código	Tipo de Pagamento	% Desconto	% Acréscimo
1	Dinheiro	10%	0%

2	Cheque	5%	0%
3 ou 4	Cartão	0%	Débito = 5% Crédito = 20%

Fonte: desenvolvido pelos autores.

Código 7

```

algoritmo "preco_final"
var preco, precofinal: real
    tipopagamento, tipocartao: inteiro
inicio
    {leitura das informações}
    escreva("Preço do produto: ")
    leia(preco)
    escreva("Tipo de pagamento: ")
    leia(tipopagamento)
    escolha(tipopagamento)
    caso 1 {pagamento em dinheiro}
        precofinal <- preco - (preco*(10/100))
        escreva("Preço final R$",
            precofinal, " (dinheiro)")
    caso 2 {pagamento em cheque}
        precofinal <- preco - preco*(5/100)
        escreva("Preço final R$ ", precofinal, " (cheque) ")
    caso 3, 4 {pagamento em cartão}
        escreva("Tipo do cartão? [1 = débito, 2 = crédito] ")
        leia(tipocartao)
        se ( tipocartao = 1 ) entao
            precofinal <- preco + (preco*(5/100) )
            escreva("Preço final R$",precofinal,"(débito)")
        senão
            se ( tipocartao = 2) então
                precofinal <- preco + (preco*(10/100) )
                escreva("Preço final R$",precofinal," (crédito)")
            senao
                escreva("0 tipo do cartão é inválido.")
        fimse
    fimse
    outrocaso {caso o tipo de pagamento seja <> 1,2,3 e 4}
        escreva("0 tipo do pagamento é inválido")
    fimescolha
fimalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução,

a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Preço do produto: 1000,00  
Tipo de pagamento: 3  
Tipo do cartão? [1 = débito, 2 = crédito] 1  
Preço final R$ 1050,00 (débito)
```

Agora que entendemos como é a estrutura de seleção múltipla, vamos aprender, no próximo tópico, as estruturas de repetição.

3. Estrutura de repetição

As estruturas de repetição são recursos utilizados na construção de algoritmos e programas e permitem ao programador definir quais blocos de instruções serão repetidos até que ocorra certa condição.

Essas estruturas têm muitas aplicações. Podemos dizer, inclusive, que qu  todos os algoritmos possuem pelo menos uma estrutura de repetição. Como exemplo, suponha a seguinte situação: você precisa criar um algoritmo que escreva dez vezes na tela um nome informado pelo usuário.

Veja, no Código 8, um possível algoritmo para solucionar esse problema.

Código 8


```
algoritmo "escrever_nome"
var
    nome: caractere
inicio
    escreva("Informe o nome: ")
    leia(nome)
    escreval("Seu nome é ", nome)
    escreval("Seu nome é ", nome)
    escreval("Seu nome é ", nome)
    escreval("Seu nome é ", nome)
    escreval("Seu nome é ", nome)
    escreval("Seu nome é ", nome)
    escreval("Seu nome é ", nome)
    escreval("Seu nome é ", nome)
    escreval("Seu nome é ", nome)
    escreval("Seu nome é ", nome)
fimalgoritmo
```

A instrução **escreva("Seu nome é", nome)** é responsável pela escrita do nome na tela. Assim, para que seja possível escrever dez vezes na tela o nome informado pelo usuário, basta repetir a instrução dez vezes.

Apesar de ser uma solução viável e que funciona perfeitamente, essa não é a melhor maneira de resolver esse problema. Além disso, imagine, por exemplo, que, em vez de dez vezes, você precisasse escrever o nome na tela 1000 vezes.



Esses tipos de problemas, nos quais um conjunto de instruções precisa ser repetido, são eficientemente solucionados pelas estruturas de repetição.

Há três tipos de estruturas de repetição:

- 1) Estrutura **para**.
- 2) Estrutura **enquanto**.
- 3) Estrutura **repita**.

A seguir, vamos compreender cada uma dessas estruturas.

Estrutura para

Antes de continuar com a leitura, assista ao vídeo para melhor compreender a estrutura de repetição **para**. No decorrer do vídeo, o professor vai evoluir o al-

goritmo da tabuada, diminuindo a quantidade de linhas programadas por meio do comando para. Não deixe de assisti-lo.

A estrutura **para** permite que o programador defina o número de vezes que um bloco de comandos deve ser repetido. O controle do número de repetições é realizado por uma variável que é conhecida como contador.

Observe, no algoritmo a seguir, a sintaxe da estrutura para:

```
para <contador> de <valor_inicial> até <valor_final> faça
    //bloco de comandos
fimpara
```

O laço de repetição da estrutura é definido pelas palavras reservadas **para ... fimpara**. Além disso, é necessário especificar uma variável do tipo inteiro, que será utilizada como contador das repetições. Essa variável deve ser previamente declarada.

O número de repetições do bloco depende dos valores definidos nos parâmetros **<valor_inicial>** e **<valor_final>**. Por exemplo, uma estrutura **para** com um laço de repetição de 100 vezes pode ser especificada da seguinte maneira:

```
para contador de 1 até 100 faça
    //bloco de comandos
fimpara
```

É importante lembrar que o ambiente para construção de algoritmos VisuAlg não utiliza acentos na especificação de comandos; assim, a estrutura **para ... de ... até ... faça** deve ser definida da seguinte maneira: **para ... de ... ate ... faca**.

No algoritmo anterior, a execução do laço de repetição ocorre da seguinte forma:

Inicialmente, a variável contador recebe o valor inicial, definido no parâmetro **<valor_inicial>**. Portanto, a variável receberá o valor 1.

Em seguida, o valor da variável **contador** é comparado ao **<valor_final>**, pois é

dessa maneira que será determinado quando as repetições devem parar. O critério de parada é simples, uma vez que o bloco de instruções será repetido quando a variável contador for menor ou igual ao valor definido no parâmetro **<valor_final>**. Essa variável, quando possuir valor maior que **<valor_final>**, fará com que o laço de repetição não seja mais executado.

Caso a variável **contador** tenha valor menor ou igual ao parâmetro **<valor_final>**, o bloco de instruções é executado. Como a execução de um algoritmo é sequencial, toda vez que o fluxo de execução chega à instrução **fimpara**, ele é remetido ao início da estrutura de repetição, ou seja, à instrução **para**.

O teste para verificar se o contador já atingiu o valor máximo é executado, e, caso já tenha alcançado o limite, a repetição termina; caso contrário, o bloco de comandos é novamente executado.

No Código 9, a estrutura de repetição **para** é utilizada para escrever, dez vezes, na tela, o nome informado pelo usuário. Nesse exemplo, a variável **inteiro (i)** foi utilizada como contador. Observe:



Código 9

```
algoritmo "estrutura_para"
//algoritmo para escrever um nome informado pelo usuário 10 vezes na tela
var
    nome: caractere
    i: inteiro    // variável contador
inicio
    escreva("Informe o nome: ")
    leia(nome)
    para i de 1 ate 10 faca
        escreva("Seu nome é ", nome)
    fimpara
finalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Informe o nome: João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
```

Veja que podemos melhorar a implementação do algoritmo anterior solicitando ao usuário o número de vezes que ele deseja repetir seu nome. Para isso, uma variável inteira deve ser declarada para receber o **valor_final** do contador. Observe:

Código 10

```
algoritmo "estrutura_para"
//algoritmo para escrever um nome informado pelo usuário n vezes na tela
var
    nome: caractere
    i: inteiro    // variável contador
    valor_final: inteiro // valor final do contador
inicio
    escreva("Informe o nome: ")
    leia(nome)
    escreva("Repetir quantas vezes? ")
    leia(valor_final)
    para i de 1 ate valor_final faca
        escreval("Seu nome é ", nome)
    fimpara
finalgoritmo
```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Informe o nome: João da Silva
Repetir quantas vezes? 3
Seu nome é João da Silva
Seu nome é João da Silva
Seu nome é João da Silva
```

É importante notar que a variável **valor_final** define o critério de parada para o laço de repetição. Quando o contador atingir o valor dessa variável, a estrutura de repetição terminará.

Da mesma maneira que podemos definir, dinamicamente, a condição de parada da estrutura de repetição por meio de uma variável para o **valor_final**, podemos, também, especificar o **valor_inicial** do contador.

O Código 11 demonstra como é possível definir a condição inicial e final da estrutura de repetição **para**. Nesse exemplo, um conjunto de números é impresso na tela em função dos valores informados pelo usuário.

Código 11



```
algoritmo "conjunto"
//algoritmo para imprimir um conjunto de valores segundo os parâmetros informados pe
var
    contador: inteiro
    valor_inicial: inteiro
    valor_final: inteiro
inicio
    escreva("Valor Inicial: ")
    leia(valor_inicial)
    escreva("Valor Final: ")
    leia(valor_final)
    escreval("Números entre ", valor_inicial, " e", valor_final)
    //estrutura de repetição
    para contador de valor_inicial ate valor_final faca
        escreva(contador)
    fimpara
fimalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Valor Inicial: 115  
Valor Final: 122  
Números entre 115 e 122  
115 116 117 118 119 120 121 122
```

Outro parâmetro interessante, mas opcional, que pode ser definido na estrutura de repetição **para** é como será o método de **incremento** ou **decremento** do contador. Por padrão, o passo incremental da estrutura **para** é definido como 1 (um). Isso significa que, a cada laço de repetição, é somado (ou incrementado) 1 (um) ao valor do contador. Também é possível fazer que seja subtraído um valor do contador. Essa operação é conhecida como **decremento**.

Para controlar como será o incremento ou o decremento do contador, é necessário acrescentar o parâmetro **passo** à estrutura. Observe:

```
para <contador> de <inicio> ate <fim> passo <valor_passo> faca  
    //bloco de comandos  
fimpara
```



Dessa forma, o parâmetro define como será o incremento ou o decremento do contador.

Quando o parâmetro **<valor_passo>** possuir valores maiores que zero, isso significará que o contador será incrementado; em contrapartida, valores menores que zero decrementam o contador.

No Código 12, demonstraremos a aplicabilidade do parâmetro **passo**. Esse algoritmo tem como objetivo escrever na tela todos os números ímpares entre 1 e 10. Veja:

Código 12

```
algoritmo "impares"
//algoritmo para imprimir os valores impares entre 1 e 10
var
    contador: inteiro
inicio
    //estrutura de repetição
    para contador de 1 ate 10 passo 2 faca
        escreva(contador)
    fimpara
fimalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

1 3 5 7 9

Vimos, a partir desses exemplos, como é aplicada a estrutura de repetição para. Mas você sabia que podemos acumular uma variável nesse tipo de estrutura? Sim, podemos! No próximo tópico, vamos aprender como os acumuladores funcionam. 

Acumulador

Quando utilizamos uma estrutura de repetição, é comum precisarmos acumular uma variável ao longo dos laços de repetição. Assim, na maioria das vezes, acumulamos a soma dessa variável. Porém, também podemos acumular a subtração, a multiplicação ou a divisão.

É importante que nos lembremos disto: quando utilizamos acumuladores, a variável usada como acumulador **sempre** será inicializada com o valor zero. Isso garante que nenhum valor prévio afete o processo do acumulador.

Para exemplificar o funcionamento do acumulador, calcularemos a soma dos números de um até cinco. Para tanto, utilizaremos uma estrutura de repetição para, e o acumulador será definido em uma variável nomeada **soma**. A aplicação de acumuladores para somar números é conhecida como **somatório**.

Como resultado da execução do algoritmo, a variável **soma** terá o valor de $1+2+3+4+5$, que é igual a 15. Acompanhe:

Código 13

```
algoritmo "acumulador"
//algoritmo para exemplificar utilização de acumuladores
var
    i: inteiro
    soma: inteiro
inicio
    //inicialização do acumulador
    /**obrigatória**
    soma <- 0
    //estrutura de repetição
    para i de 1 ate 5 passo 1 faca
        soma <- soma + i
    fimpara
    escreva("Soma = ", soma)
finalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução a seguir. As informações utilizadas como entrada são apenas ilustrativas.



Resultado da execução do algoritmo:

```
Soma = 15
```

No Código 14, observe outro exemplo, no qual utilizaremos um acumulador para somar um conjunto de cinco valores reais informados pelo usuário. Ao final, exibiremos a soma desses números e, também, a média obtida.

Código 14


```

algoritmo "acumulador"
//algoritmo para exemplificar a utilização de acumuladores para números reais
var
    i: inteiro    //contador
    numero: real  //número fornecido pelo usuário
    soma: real    //acumulador
    media: real
inicio
    soma <- 0
    para i de 1 ate 5 passo 1 faca
        escreva("Informe um número: ")
        leia(numero)
        soma <- soma + numero
    fimpara
    media <- soma / (i-1)
    escreval("Soma = ", soma)
    escreval("Média = ", media)
Fimalgoritmo

```

Fique atento a algumas considerações em relação ao algoritmo do Código 14. O acumulador **soma** é utilizado para armazenar a soma dos números informados pelo usuário.

A média dos números é calculada por meio da divisão da soma (resultado do acumulador) pelo valor da variável **contador i** menos 1 (um). 

Resultado da execução do algoritmo:

```

Informe um número: 2,8
Informe um número: 5,4
Informe um número: 3,2
Informe um número: 7,8
Informe um número: 9,6
Soma = 28.8
Média = 7.2

```

Estrutura enquanto

A estrutura de repetição **enquanto** é outra maneira de produzir um laço de repetição em um algoritmo. Essa estrutura permite ao programador especificar a repetição de um bloco de comandos enquanto uma condição permanecer verdadeira.

Ao contrário da estrutura **para**, que pode ser aplicada apenas em problemas envolvendo contadores, a estrutura de repetição **enquanto** pode ser utilizada para laços de repetição com ou sem contadores. Isso permite que a estrutura enquanto seja utilizada em problemas nos quais não sabemos o número de vezes que o bloco de comandos será repetido.

A sintaxe da estrutura de repetição **enquanto** é definida a seguir:

```
enquanto <expressão_lógica> faca  
  //bloco de comandos  
fimenquanto
```

A expressão lógica especificada na estrutura define a condição de parada dos laços de repetição. Para entender melhor a estrutura **enquanto**, observe sua interpretação no exemplo a seguir: enquanto uma determinada condição for verdadeira, repetir o bloco de comandos.

Veja que podemos aplicar a estrutura **enquanto** em qualquer tipo de problema resolvido pela estrutura **para**. Por exemplo: considere uma estrutura **para**  paz de exibir o nome **João da Silva** dez vezes na tela.

Observe a solução utilizando a estrutura **para**:

```
para contador de 1 ate 10 faca  
  escreva("João da Silva")  
fimpara
```

Agora, observe a solução utilizando a estrutura **enquanto**:

```
contador <- 1  
enquanto (contador <= 10) faca  
  escreva("João da Silva")  
  contador <- contador + 1  
fimenquanto
```

Antes prosseguir com seus estudos em relação à estrutura de repetição, responda à questão a seguir para testar se, de fato, está compreendendo esta lógica de programação.

Para aplicar a estrutura **enquanto** em problemas que envolvem a repetição utilizando contadores, você deve se lembrar de inicializar a variável contador. Além disso, a cada passo da repetição, é preciso incrementar ou decrementar o contador é "**contador <- contador + 1**".

Apesar de a solução que utiliza a estrutura **enquanto** possuir um número maior de linhas do que a solução que utiliza **para**, existem centenas de problemas em que a estrutura **enquanto** é mais vantajosa. Também é importante salientar que inúmeros problemas resolvidos pela estrutura enquanto não podem ser solucionados utilizando a estrutura **para**.

Vejamos, agora, um exemplo da utilização da estrutura enquanto para a repetição de blocos de comandos **sem** usar contadores. Nesse problema, elaboraremos um algoritmo que calcule o quadrado de um número informado pelo usuário indefinidas vezes. O cálculo só terminará quando o número informado pelo usuário for igual a zero.

Código 15



```
algoritmo "enquanto"
//algoritmo para exemplificar a utilização da estrutura enquanto sem contadores
var
    num: inteiro
inicio
    num <- -1
    enquanto ( num <> 0 ) faca
        escreva("Informe um número: ")
        leia(num)
        se (num <> 0 ) entao
            escreval("Quadrado de", num," é igual a", (num*num))
        fimse
    fimenquanto
fimalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Informe um número: 2
Quadrado de 2 é igual a 4
Informe um número: 4
Quadrado de 4 é igual a 16
Informe um número: 6
Quadrado de 6 é igual a 36
Informe um número: 0
Fim da execução.
```

É importante notar que, no exemplo anterior, nenhuma variável foi utilizada como contador. Outro fato interessante é a atribuição do valor -1 à variável **num**. Essa instrução garante que a estrutura de repetição será executada pelo menos uma vez, pois, como o valor padrão da variável **num** é igual a zero, não fosse o valor -1, a estrutura de repetição nunca seria executada.

Devemos atentar também para a condição de parada da estrutura de repetição (**num <> 0**). Note que essa condição significa: enquanto o número informado pelo usuário for diferente de zero, executar o bloco de comandos.

No próximo exemplo, utilizaremos a estrutura enquanto em um laço de repetição com contador. Esse algoritmo tem como objetivo realizar a leitura dos dados de cinco pessoas (**M = masculino e F = feminino**). Em seguida, ele deverá exibir na tela o número de pessoas do sexo masculino e o número de pessoas do sexo feminino.

Código 16

```

algoritmo "enquanto_com_contador"
//algoritmo para exemplificar a utilização da estrutura enquanto com contadores
var
    contador : inteiro    //contador
    sexo: caractere
    totalmasculino: inteiro
    totalfeminino: inteiro
inicio
    totalmasculino <- 0
    totalfeminino <- 0
    contador <- 1
    enquanto ( contador <= 5 ) faça
        escreva("Sexo [M=masculino; F=feminino]: ")
        leia(sexo)
        se ( sexo = "M" ) entao
            totalmasculino <- totalmasculino +1
        senao
            se ( sexo = "F" ) entao
                totalfeminino <- totalfeminino +1
            fimse
        fimse
        contador <- contador + 1
    fimenquanto
    escreval("Total do sexo masculino: ", totalmasculino)
    escreval("Total do sexo feminino: ", totalfeminino)
fimalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```

Sexo [M=masculino; F=feminino]: M
Sexo [M=masculino; F=feminino]: M
Sexo [M=masculino; F=feminino]: F
Sexo [M=masculino; F=feminino]: M
Sexo [M=masculino; F=feminino]: M
Total do sexo masculino: 4
Total do sexo feminino: 1

```

Entendemos, por meio dos exemplos expostos, a estrutura **enquanto**; na sequência, veremos a última estrutura que compõe as estruturas de repetição.

Estrutura repita

Na estrutura **repita**, o bloco de comandos é repetido até que uma determinada condição se torne verdadeira. Ao contrário da estrutura de repetição **enquanto**, em que a condição de parada é testada no início da estrutura, na estrutura **repita**, a condição de parada é testada no final.

A sintaxe dessa estrutura é definida como se segue:

```
repita  
  
    //bloco de comandos  
  
ate <expressão_lógica>
```

A expressão lógica define a condição de parada da estrutura; assim, o bloco de comandos é repetido até que a condição seja satisfeita.

Da mesma maneira que na estrutura **enquanto**, a estrutura **repita** permite que conectivos lógicos sejam utilizados na condição de parada, como, por exemplo: **repita ... até (condicao1 ou condicao2)**.



Para demonstrar a utilização da estrutura **repita**, escreveremos um algoritmo capaz de escrever na tela os números pares entre um e dez. Observe:

Código 17

```
algoritmo "estrutura_repita"  
//algoritmo para demonstrar a utilização da estrutura repita  
var  
    i: inteiro  
inicio  
    i <- 1  
    repita  
        //verifica se o número i é par  
        se ( ( i % 2 ) = 0 ) entao  
            escreva(i)  
        fimse  
        //incrementa o contador  
        i <- i + 1  
    ate ( i = 10 )  
fimalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
2 4 6 8
```

Note que, no Código 17, o bloco de comandos é repetido até que a condição seja satisfeita; nesse caso, **i=10**. Como a condição de parada é testada no final da estrutura, quando i atinge o valor 10, o laço de repetição termina.

Outro fato interessante é a forma utilizada para determinar se um número é par. Para isso, a seguinte condição foi necessária: $((i \% 2) = 0)$. Isso significa que, se o resto da divisão de i por 2 for igual a zero, então, esse número i é par. Para determinar números ímpares, basta trocarmos o operador de igualdade pelo operador de diferença (<>).

Veja, no Código 18, mais um exemplo que demonstra a utilização da estrutura **repita** sem contadores. Nesse algoritmo, o objetivo é receber as idades de um conjunto indeterminado de pessoas até que uma idade negativa seja informada. Ao final, deve-se apresentar na tela a quantidade de pessoas nas faixas etárias: (a) 1-15 anos; (b) 16-30 anos; (c) 31-45 anos; (d) 46-60 anos; (e) maiores de 60 anos.

Código 18

```

algoritmo "estrutura_repita"
//algoritmo para demonstrar a utilização da estrutura repita
var
    idade: inteiro
    continuar: caractere
    faixa1: inteiro    {1 - 15 anos}
    faixa2: inteiro    {16 - 30 anos}
    faixa3: inteiro    {31 - 45 anos}
    faixa4: inteiro    {46 - 60 anos}
    faixa5: inteiro    {maior que 60 anos}
inicio
    //inicializar contadores
    faixa1 <- 0
    faixa2 <- 0
    faixa3 <- 0
    faixa4 <- 0
    faixa5 <- 0
    repita
        escreva("Idade: ")
        leia(idade)
        se ((idade >=0) e (idade <=15)) entao
            faixa1 <- faixa1 + 1
        senao
            se ((idade >=16) e (idade <=30)) entao
                faixa2 <- faixa2 + 1
            senao
                se ((idade >=31) e (idade <=45)) entao
                    faixa3 <- faixa3 + 1
                senao
                    se ((idade >=46) e (idade <=60)) entao
                        faixa4 <- faixa4 + 1
                    senao
                        se (idade > 60) entao
                            faixa5 <- faixa5 + 1
                        fimse
                    fimse
                fimse
            fimse
        fimse
        escreva("Deseja continuar (S ou N)?" )
        leia(continuar)
    ate ( (continuar = "N") ou (continuar = "n") )
    escreval("Número pessoas 1 - 15 anos: ", faixa1)
    escreval("Número pessoas 16 - 30 anos: ", faixa2)
    escreval("Número pessoas 31 - 45 anos: ", faixa3)
    escreval("Número pessoas 46 - 60 anos: ", faixa4)
    escreval("Número pessoas > 60 anos: ", faixa5)
fimalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução,

a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Início da execução
Idade: 10
Deseja continuar (S ou N)?s
Idade: 20
Deseja continuar (S ou N)?s
Idade: 30
Deseja continuar (S ou N)?s
Idade: 40
Deseja continuar (S ou N)?s
Idade: 50
Deseja continuar (S ou N)?s
Idade: 60
Deseja continuar (S ou N)?s
Idade: 70
Deseja continuar (S ou N)?n
Número pessoas 1 - 15 anos: 1
Número pessoas 16 - 30 anos: 2
Número pessoas 31 - 45 anos: 1
Número pessoas 46 - 60 anos: 2
Número pessoas > 60 anos: 1
Fim da execução.
```



Chegou o momento de praticar os conceitos estudados, a partir dos exercícios propostos a seguir.

4. Exercícios resolvidos

Neste tópico, você encontrará alguns exercícios resolvidos e comentados que facilitarão sua compreensão e o auxiliarão em sua aprendizagem – afinal, eles possibilitam que você esclareça suas dúvidas. Portanto, não deixe de resolvê-los e, caso tenha dúvidas, entre em contato com o seu tutor.

Exercício 1

Elabore um algoritmo que receba dois números e apresente na tela o maior deles.

Código 19

```

algoritmo "maior_numero"
//algoritmo para determinar o maior número
var
    num1, num2: real
inicio
    escreva("1º Número: ")
    leia(num1)
    escreva("2º Número: ")
    leia(num2)
    se ( num1 > num2) entao
        escreva(num1, " é maior que ", num2)
    fimse
    se ( num2 > num1) entao
        escreva(num2, " é maior que ", num1)
    fimse
    se ( num1 = num2) entao
        escreva(num1, " é igual a ", num2)
    fimse
fimalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:



```

1º Número: 10
2º Número: 15
15 é maior que 10

```

Exercício 2

Escreva um algoritmo que receba dois números, executando a operação desejada, conforme a Tabela 1.

Tabela 1 Tabela de operações.

Escolha	Operação
1	Soma dos números
2	Subtração dos números
3	Produto entre os números
4	Divisão dos números

Código 20

```
algoritmo "opcoes_usuario"
var
    num1, num2: real
    operacao: inteiro
inicio
    escreva("1º Número: ")
    leia(num1)
    escreva("2º Número: ")
    leia(num2)
    escreva("Operação [1,2,3 ou 4]: ")
    leia(operacao)
    escolha(operacao)
    caso 1
        escreva(num1," + ",num2," =", (num1+num2) )
    caso 2
        escreva(num1," - ",num2," =", (num1-num2) )
    caso 3
        escreva(num1," * ",num2," =", (num1*num2) )
    caso 4
        se ( num2 <> 0 ) entao
            escreva(num1," / ",num2," = ", (num1/num2) )
        senao
            escreva("Divisão por zero.")
        fimse
    outrocaso
        escreva("Opção inválida.")
    fimescolha
fimalgoritmo
```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
1º Número: 10
2º Número: 0
Operação [1,2,3 ou 4]: 4
Divisão por zero.
```

Exercício 3

Escreva um algoritmo que analise os dados de uma pesquisa realizada com os habitantes da cidade de Taboão do Norte. Os dados coletados foram: idade, sexo (M/F) e salário. Com a pesquisa, deseja-se saber: (a) média do salário dos habitantes; (b) maior e menor idade do grupo; (c) quantidade de mulheres com salários superiores a R\$ 600,00.

A execução do algoritmo termina quando uma idade negativa for informada.

Código 21



```

algoritmo "pesquisa"
//algoritmo para analisar os dados de uma pesquisa realizada com os habitantes de Ta
var
    idade: inteiro
    sexo: caractere
    salario: real
    soma: real    {total de salários}
    maior,menor: inteiro {maior e menor idade}
    contador: inteiro
    feminino: inteiro
inicio
    {inicialização das variáveis}
    idade <- 0
    soma <- 0
    contador <- 0
    feminino <- 0
    enquanto ( idade >= 0 ) faca
        {leitura das informações}
        escreva("Idade: ")
        leia(idade)
        escreva("Sexo (M ou F): ")
        leia(sexo)
        escreva("Salário: ")
        leia(salario)
        se ( idade >= 0 ) entao
            {soma dos salários}
            soma <- soma + salario
            {primeira idade informada pelo usuário}
            se ( contador = 0 ) entao
                maior <- idade
                menor <- idade
            senao
                {determinar maior idade}
                se ( idade > maior ) entao
                    maior <- idade
                fimse
                {determinar menor idade}
                se ( idade < menor ) entao
                    menor <- idade
                fimse
            fimse
            se (((sexo="F") ou (sexo="f")) e (salario>600)) entao
                feminino <- feminino + 1
            fimse
            contador <- contador + 1
            {incrementar o contador}
        fimse
    fimenquanto
    {resultados}
    escreval("Média dos salários R$ ", (soma/contador) )
    escreval("Maior idade = ", maior)
    escreval("Menor idade = ", menor)

```



```
escreval("Sexo feminino e salários superiores a R$ 600,00 :", feminino)
finalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Idade: 10
Sexo (M ou F): f Salário: 800

Idade: 20
Sexo (M ou F): m Salário: 200

Idade: -1
Sexo (M ou F): f Salário: 0

Média dos salários R$ 500,00
Maior idade = 20
Menor idade = 10
Sexo feminino e salários superiores a R$ 600,00 :1
```



Exercício 4

Observe o cardápio de uma lanchonete na Tabela 2:

Tabela 2 Cardápio de uma lanchonete.

Código	Produto	Preço
100	Cachorro Quente	R\$ 3,50
101	Hambúrguer	R\$ 4,00
102	X-Tudo	R\$ 5,00
103	Refrigerante	R\$ 2,50
104	Suco	R\$ 1,50

Fonte: desenvolvida pelos autores.

Analisando o cardápio da lanchonete, escreva um algoritmo que leia o código dos itens pedidos e as quantidades desejadas. O algoritmo deve, ainda, calcu-

lar e mostrar o valor a ser pago por item, bem como o total geral do pedido. O cliente deverá informar quando o pedido deve ser encerrado (adaptado de ASCENCIO; CAMPOS, 2008).

Código 22

```
algoritmo "lanchonete"
//algoritmo para simular o funcionamento de uma lanchonete
var
    codigo: inteiro
    quant: inteiro
    totalpedido: real
    valoritem : real
    sair: caractere
inicio
    totalpedido <- 0
    repita
        escreva("Código: ")
        leia(codigo)
        escreva("Quantidade: ")
        leia(quant)
        valoritem <- 0
        escolha(codigo)
        caso 100
            valoritem <- quant*3.50
            escreval(quant, " Cachorro Quente = ", valoritem )
        caso 101
            valoritem <- quant*4.00
            escreva(quant, " Hambúrguer = ", valoritem )
        caso 102
            valoritem <- quant*5.00
            escreval(quant, " X-Tudo = ", valoritem )
        caso 103
            valoritem <- quant*2.50
            escreval(quant, " Refrigerante = ", valoritem )
        caso 104
            valoritem <- quant*1.50
            escreval(quant, " Suco = ", valoritem )
        outrocaso
            escreval("0 código informado é inválido.")
        fimescolha
    //total pedido
    totalpedido <- totalpedido + valoritem
    escreva("Finalizar pedido (S ou N)? ")
    leia(sair)
    ate ( (sair = "S") ou (sair = "s") )
    escreval("Total do pedido R$ ", totalpedido)
finalgoritmo
```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Código: 100
Quantidade: 2
2 Cachorro Quente = 7,0
Finalizar pedido (S ou N)? n

Código: 104
Quantidade: 2
2 Suco = 3,0
Finalizar pedido (S ou N)? s

Total do pedido R$ 10,0
```

Exercício 5

Escreva um algoritmo que receba as seguintes informações de uma pessoa: (i) nome completo; (ii) idade; (iii) sexo; e (iv) estado civil. Para o nome da pessoa, você deve utilizar uma variável do tipo caractere; para a idade, uma variável do tipo inteiro. O sexo da pessoa deve ser informado por meio de um caractere, sendo M para masculino e F para feminino. O estado civil também deve ser informado utilizando um caractere, o qual poderá ter as seguintes opções: C para casado(a); S para solteiro(a); D para divorciado(a) e V para viúvo(a). Ao final, o programa deverá exibir na tela as informações detalhadas da pessoa, por exemplo:

- Nome: João da Silva
- Idade: 36 anos
- Sexo: Masculino
- Estado Civil: Solteiro

Código 23


```

algoritmo "informacoes_pessoais"
var
    nome_completo: caractere
    idade: inteiro
    sexo: caractere
    estado_civil: caractere
    sexo_extenso: caractere
    estado_civil_extenso: caractere
inicio
    escreval("INFORMAÇÕES PESSOAIS")
    escreval("")
    escreva("Informe o nome completo: ")
    leia(nome_completo)
    escreva("Informe a idade: ")
    leia(idade)
    escreva("Informe o sexo [M]masculino ou [F]feminino: ")
    leia(sexo)
    escreval("Informe o estado civil [C]casado(a), ")
    escreval("[S]solteiro(a), ")
    escreval("[D]divorciado(a) ou ")
    escreva("[V]viúvo(a): ")
    leia(estado_civil)
    se ( (sexo = "M") ou (sexo = "m")) entao
        sexo_extenso <- "Masculino"
    senao
        se ( (sexo = "F") ou (sexo = "f")) entao
            sexo_extenso <- "Feminino"
        fimse
    fimse
    se ( (estado_civil = "C") ou (estado_civil = "c")) entao
        estado_civil_extenso <- "Casado(a)"
    senao
        se ( (estado_civil = "S") ou (estado_civil = "s")) entao
            estado_civil_extenso <- "solteiro(a)"
        senao
            se ( (estado_civil = "D") ou (estado_civil = "d")) entao
                estado_civil_extenso <- "Divorciado(a)"
            senao
                se ( (estado_civil = "V") ou (estado_civil = "v")) entao
                    estado_civil_extenso <- "Divorciado(a)"
                fimse
            fimse
        fimse
    fimse
    escreval("")
    escreval("RESULTADOS")
    escreval("Nome: ", nome_completo)
    escreval("Idade: ", idade, " anos")
    escreval("Sexo: ", sexo_extenso)
    escreval("Estado Civil: ", estado_civil_extenso)
fimalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

INFORMAÇÕES PESSOAIS

Informe o nome completo: Carlos Antônio
Informe a idade: 34
Informe o sexo [M]masculino ou [F]feminino: m
Informe o estado civil
[C]casado(a),
[S]solteiro(a),
[D]divorciado(a) ou
[V]viúvo(a): c

RESULTADOS

Nome: Carlos Antônio
Idade: 34 anos
Sexo: Masculino
Estado Civil: Casado(a)



Exercício 6

Uma empresa de seguros comercializa seguros de vida e necessita que seja elaborado um algoritmo capaz de determinar o risco de venda de uma apólice de seguros. Esse risco é determinado em função da faixa etária da pessoa, e as seguintes regras são adotadas: (i) idade entre 18 e 24 anos tem risco baixo; (ii) idade superior a 24 anos e inferior ou igual a 40 anos tem risco médio; e (iii) idade superior a 40 anos e inferior ou igual a 70 anos tem risco alto. Para todos os casos fora dessas regras, não é possível a aquisição de uma apólice de seguro. Elabore um algoritmo capaz de resolver esse problema.

Código 24

```

algoritmo "risco_venda_apolice_seguro"
var
    idade: inteiro
    resultado_risco: caractere
inicio
    escreval("SEGUROS - RISCO DE VENDA")
    escreval("")
    escreva("Informe a idade: ")
    leia(idade)
    se ( (idade >= 18) e (idade <= 24) ) entao
        resultado_risco <- "0 risco de venda de uma apólice de seguros para essa pessoa"
    senao
        se ( (idade > 24) e (idade <= 40) ) entao
            resultado_risco <- "0 risco de venda de uma apólice de seguros para essa pessoa"
        senao
            se ( (idade > 40) e (idade <= 70) ) entao
                resultado_risco <- "0 risco de venda de uma apólice de seguros para essa pessoa"
            senao
                resultado_risco <- "Para essa faixa etária NÃO são comercializadas apólices de seguro."
            fimse
        fimse
    fimse
    fimse
    escreval("")
    escreval("RESULTADOS")
    escreval(resultado_risco)
fimalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```

SEGUROS - RISCO DE VENDA

Informe a idade: 15

RESULTADOS

Para essa faixa etária NÃO são comercializadas apólices de seguro.

```

Exercício 7

Uma empresa necessita realizar um levantamento dos bens patrimoniais de informática existentes. Para isso, você deve elaborar um algoritmo que solicite ao usuário o tipo de bem patrimonial, sendo: C para computador; I para im-

pressora; e S para scanner. Em seguida, a seguinte mensagem deverá ser exibida: Deseja continuar o levantamento [S] ou [N]?. Caso o usuário escolha S (sim), o algoritmo deverá solicitar novamente o tipo de bem patrimonial. Caso o usuário escolha N (não), o algoritmo deverá exibir o resultado do levantamento. O resultado deverá conter o número de equipamentos de cada tipo.

Código 25



```

algoritmo "levantamento_patrimonial"
var
    tipo_bem_patrimonial: caractere
    continuar: caractere
    contador_computador: inteiro
    contador_impresora: inteiro
    contador_scanner: inteiro
inicio
    contador_computador <- 0
    contador_impresora <- 0
    contador_scanner <- 0
    repita
        escreval(" ")
        escreval("--- LEVANTAMENTO DE BENS PATRIMONIAIS ---")
        escreval(" ")
        escreval("")
        escreval("Bens Patrimoniais: ")
        escreval(".....[C] COMPUTADOR")
        escreval(".....[I] IMPRESSORA")
        escreval(".....[S] SCANNER")
        escreva("Selecione o tipo do bem: ")
        leia(tipo_bem_patrimonial)
        se ( (tipo_bem_patrimonial = "C") ou (tipo_bem_patrimonial = "c") ) entao
            contador_computador <- contador_computador + 1
        senao
            se ( (tipo_bem_patrimonial = "I") ou (tipo_bem_patrimonial = "i")) entao
                contador_impresora <- contador_impresora + 1
            senao
                se ( (tipo_bem_patrimonial = "S") ou (tipo_bem_patrimonial = "s")) entao
                    contador_scanner <- contador_scanner + 1
                fimse
            fimse
        fimse
        escreva("Deseja continuar o levantamento [S] ou [N]?")
        leia(continuar)
    ate ( (continuar = "N") ou (continuar="n"))
    escreval("")
    escreval("RESULTADOS")
    escreval("Total de bens patrimoniais do tipo COMPUTADOR: ", contador_computador)
    escreval("Total de bens patrimoniais do tipo IMPRESSORA: ", contador_impresora)
    escreval("Total de bens patrimoniais do tipo SCANNER: ", contador_scanner)
finalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

--- LEVANTAMENTO DE BENS PATRIMONIAIS ---

Bens Patrimoniais:

.....[C] COMPUTADOR
.....[I] IMPRESSORA
.....[S] SCANNER

Selecione o tipo do bem: c

Deseja continuar o levantamento [S] ou [N]?s

--- LEVANTAMENTO DE BENS PATRIMONIAIS ---

Bens Patrimoniais:

.....[C] COMPUTADOR
.....[I] IMPRESSORA
.....[S] SCANNER

Selecione o tipo do bem: c

Deseja continuar o levantamento [S] ou [N]?s

--- LEVANTAMENTO DE BENS PATRIMONIAIS ---

Bens Patrimoniais:

.....[C] COMPUTADOR
.....[I] IMPRESSORA
.....[S] SCANNER

Selecione o tipo do bem: s

Deseja continuar o levantamento [S] ou [N]?s

--- LEVANTAMENTO DE BENS PATRIMONIAIS ---

Bens Patrimoniais:

.....[C] COMPUTADOR
.....[I] IMPRESSORA
.....[S] SCANNER

Selecione o tipo do bem: i

Deseja continuar o levantamento [S] ou [N]?s

--- LEVANTAMENTO DE BENS PATRIMONIAIS ---

Bens Patrimoniais:

.....[C] COMPUTADOR
.....[I] IMPRESSORA



```
.....[S] SCANNER
```

```
Selecione o tipo do bem: s
```

```
Deseja continuar o levantamento [S] ou [N]?n
```

```
RESULTADOS
```

```
Total de bens patrimoniais do tipo COMPUTADOR: 2
```

```
Total de bens patrimoniais do tipo IMPRESSORA: 1
```

```
Total de bens patrimoniais do tipo SCANNER: 2
```

Exercício 8

Uma empresa de fornecimento de energia elétrica realiza, mensalmente, a leitura dos medidores de consumo. Essas informações são dispostas em uma ficha, que contém:

I – Código do consumidor.

II – Quantidade de kW/h consumida durante o mês.

III – Tipo do consumidor, sendo: (1) residencial – em que o preço por kW/h é de R\$ 5,68; (2) comercial – em que o preço por kW/h é de R\$ 8,25; e (3) industrial – em que o preço por kW/h é de R\$ 10,32.



Escreva um algoritmo computacional que seja capaz de:

- Calcular e exibir na tela o custo para cada consumidor.
- Calcular e exibir na tela o total de consumo para os três tipos de consumidor.
- Calcular e exibir na tela a média total de consumo dos tipos 1, 2 e 3.

Código 26

```
algoritmo "empresa_energia_eletrica"
```

```
var
```

```
    codigo_consumidor: inteiro  
    quantidade_kwh: real  
    tipo_consumidor: inteiro  
    custo_consumidor : real  
    custo_total_consumidor1: real  
    custo_total_consumidor2: real  
    custo_total_consumidor3: real  
    custo_total: real  
    numero_total_consumidores1: inteiro  
    numero_total_consumidores2: inteiro  
    numero_total_consumidores3: inteiro  
    media_total_consumidor1: real  
    media_total_consumidor2: real  
    media_total_consumidor3: real  
    continuar: caractere
```

```
inicio
```

```
    custo_total_consumidor1 <- 0  
    custo_total_consumidor2 <- 0  
    custo_total_consumidor3 <- 0  
    numero_total_consumidores1 <- 0  
    numero_total_consumidores2 <- 0  
    numero_total_consumidores3 <- 0
```

```
    repita
```

```
        escreval("      ")
```

```
        escreval("-EMPRESA DE ENERGIA ELÉTRICA  --")
```

```
        escreval("      ")
```

```
        escreval("")
```

```
        escreva("Informe o código do consumidor: ")
```

```
        leia(codigo_consumidor)
```

```
        escreva("Informe quantidade de kWh consumida durante o mês: ")
```

```
        leia(quantidade_kwh)
```

```
        escreval("Informe o tipo do consumidor: ")
```

```
        escreval(".....[1] RESIDENCIAL")
```

```
        escreval(".....[2] COMERCIAL")
```

```
        escreval(".....[3] INDUSTRIAL")
```

```
        escreva(".....Tipo: ")
```

```
        leia(tipo_consumidor)
```

```
        custo_consumidor <- 0
```

```
        se ( tipo_consumidor = 1 ) entao
```

```
            custo_consumidor <- quantidade_kwh * 5.68
```

```
            custo_total_consumidor1 <- custo_total_consumidor1 + custo_consumidor
```

```
            numero_total_consumidores1 <- numero_total_consumidores1 + 1
```

```
        senao
```

```
            se ( tipo_consumidor = 2 ) entao
```

```
                custo_consumidor <- quantidade_kwh * 8.25
```

```
                custo_total_consumidor2 <- custo_total_consumidor2 + custo_consumidor
```

```
                numero_total_consumidores2 <- numero_total_consumidores2 + 1
```

```
            senao
```

```
                se ( tipo_consumidor = 3 ) entao
```

```
                    custo_consumidor <- quantidade_kwh * 10.32
```




```

        custo_total_consumidor3 <- custo_total_consumidor3 + custo_consumidor
        numero_total_consumidores3 <- numero_total_consumidores3 + 1
    fimse
Fimse
Fimse
escreval("")
escreval("----- RELATÓRIO DO CONSUMIDOR ")
escreval("-- Código do Consumidor: ", codigo_consumidor)
escreval("-- Custo Total R$ ", custo_consumidor:4:2 )
escreval("")
escreva("Deseja continuar [S] ou [N]?")
leia(continuar)
ate ( (continuar = "N") ou (continuar="n") )
custo_total <- custo_total_consumidor1+custo_total_consumidor2+custo_total_consumidor3
media_total_consumidor1 <- custo_total_consumidor1 / numero_total_consumidores1
media_total_consumidor2 <- custo_total_consumidor2 / numero_total_consumidores2
media_total_consumidor3 <- custo_total_consumidor3 / numero_total_consumidores3
escreval(" ")
escreval("----- RELATÓRIO DE FINAL ")
escreval(" ")
escreval("")
escreval("-- TOTAL DE CONSUMO por Tipo de Consumidor: ")
escreval(".....[1] RESIDENCIAL.. R$ ", custo_total_consumidor1:4:2)
escreval(".....[2] COMERCIAL..R$ ", custo_total_consumidor2:4:2)
escreval(".....[3] INDUSTRIAL..R$ ", custo_total_consumidor3:4:2)
escreval("  TOTAL.....R$ ", custo_total:4:2)
escreval("")
escreval("-- MÉDIA DE CONSUMO por Tipo de Consumidor: ")
escreval(".....[1] RESIDENCIAL.. R$ ", media_total_consumidor1:4:2)
escreval(".....[2] COMERCIAL...R$ ", media_total_consumidor2:4:2)
escreval(".....[3] INDUSTRIAL....R$ ", media_total_consumidor3:4:2)
escreval(" ")
fimalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo

----EMPRESA DE ENERGIA ELÉTRICA -----

Informe o código do consumidor: 12

Informe quantidade de kWh consumida durante o mês: 320 Informe o tipo do consumidor:

.....[1] RESIDENCIAL

.....[2] COMERCIAL

.....[3] INDUSTRIAL

.....Tipo: 1

----- RELATÓRIO DO CONSUMIDOR -----

-- Código do Consumidor: 12

-- Custo Total R\$ 1817.60 Deseja continuar [S] ou [N]?s

----EMPRESA DE ENERGIA ELÉTRICA -----

Informe o código do consumidor: 18

Informe quantidade de kWh consumida durante o mês: 1250

Informe o tipo do consumidor:

.....[1] RESIDENCIAL

.....[2] COMERCIAL

.....[3] INDUSTRIAL

.....Tipo: 3

----- RELATÓRIO DO CONSUMIDOR -----

-- Código do Consumidor: 18

-- Custo Total R\$ 12900.00 Deseja continuar [S] ou [N]?s

----EMPRESA DE ENERGIA ELÉTRICA -----

Informe o código do consumidor: 14

Informe quantidade de kWh consumida durante o mês: 280 Informe o tipo do consumidor:

.....[1] RESIDENCIAL

.....[2] COMERCIAL

.....[3] INDUSTRIAL

.....Tipo: 1

----- RELATÓRIO DO CONSUMIDOR -----

-- Código do Consumidor: 14

-- Custo Total R\$ 1590.40

Deseja continuar [S] ou [N]?s

----EMPRESA DE ENERGIA ELÉTRICA -----

Informe o código do consumidor: 16

Informe quantidade de kWh consumida durante o mês: 400 Informe o tipo do consumidor:

.....[1] RESIDENCIAL



```

.....[2] COMERCIAL
.....[3] INDUSTRIAL
.....Tipo: 2

----- RELATÓRIO DO CONSUMIDOR -----

-- Código do Consumidor: 16
-- Custo Total R$ 3300.00 Deseja continuar [S] ou [N]?s

---EMPRESA DE ENERGIA ELÉTRICA -----

Informe o código do consumidor: 19
Informe quantidade de kWh consumida durante o mês: 200 Informe o tipo do consumidor:

.....[1] RESIDENCIAL
.....[2] COMERCIAL
.....[3] INDUSTRIAL
.....Tipo: 2

----- RELATÓRIO DO CONSUMIDOR -----

-- Código do Consumidor: 19
-- Custo Total R$ 1650.00

Deseja continuar [S] ou [N]?n

RELATÓRIO DE FINAL

-- TOTAL DE CONSUMO por Tipo de Consumidor:

.....[1] RESIDENCIAL .. R$ 3408.00
.....[2] COMERCIAL .....R$ 4950.00
.....[3] INDUSTRIAL ....R$ 12900.00

.....TOTAL .....R$ 21258.00

-- MÉDIA DE CONSUMO por Tipo de Consumidor:

.....[1] RESIDENCIAL .. R$ 1704.00
.....[2] COMERCIAL .....R$ 2475.00
.....[3] INDUSTRIAL ....R$ 12900.00

```



Após realizar os exercícios apresentados anteriormente, para finalizar os estudos desta unidade, faça as leituras indicadas a seguir.



Vamos aprofundar ainda mais nossos conhecimentos sobre esse assunto?

Para isso, sugerimos que faça uma leitura atenta dos seguintes capítulos:

- Capítulo 8 – ***Estrutura de repetição*** e Capítulo 9 – *Estrutura de decisão*, presentes na obra *Linguagem e lógica de programação* (ALVEZ, 2014).
- Capítulo 4 – *Programação com decisão* e Capítulo 5 – *Programação com laços*, da obra *Algoritmos – Lógica para Desenvolvimento de Programação de Computadores* (MANZANO; OLIVEIRA, 2019). Essas obras estão disponíveis na Minha Biblioteca.

5. Considerações

Nesta unidade, você teve a oportunidade de estudar as estruturas de controle e perceber que elas controlam o fluxo de execução dos algoritmos. Por meio das estruturas condicionais, podemos especificar condições para que determinadas partes do algoritmo sejam executadas, e, por meio das estruturas de repetição, é possível executar várias vezes determinados blocos de comandos.



Na próxima unidade, trataremos das estruturas de dados.



Unidade 4 – Estruturas de Dados

Objetivos

- Organizar informações por meio de estruturas de dados.
- Manipular grandes volumes de informações por meio de estruturas homogêneas.

Conteúdos

- **Estruturas de dados** homogêneas.
- Vetores.
- Matrizes.

Problematização

Qual a diferença entre vetores e matrizes? Como trabalhar com vetores e matrizes? Como utilizar loops de repetição para manipular dados em vetores e matrizes?



Orientações para o estudo

Esta unidade está estruturada para que você compreenda como organizar e manipular as informações pela aplicação da teoria de estrutura de dados. Como é de praxe, além de realizar as leituras indicadas e assistir ao vídeo para aprofundamento dos conteúdos que serão estudados, responda às questões propostas para verificar sua aprendizagem. Estamos na reta final dos estudos desta disciplina, por isso, mantenha sua atenção para um melhor aproveitamento!

Bons estudos!

1. Introdução

Nesta unidade, você terá a oportunidade de aprender as principais estruturas de dados existentes para a construção de algoritmos e programas.

Como já vimos no decorrer do estudo das unidades anteriores, a principal função de um algoritmo é a manipulação de dados. Até o momento, o meio que utilizamos para essa manipulação foram as variáveis. No entanto, à medida que os algoritmos crescem no tamanho e na complexidade, manipularmos as informações por meio das variáveis se torna inviável.

Suponha o seguinte problema: como elaborar um algoritmo que calcule a média de preço entre cinco produtos fornecidos pelo usuário?

Como você teve possibilidade de observar, essa tarefa pode ser facilmente executada utilizando-se uma estrutura de repetição e uma variável acumuladora. Ao final do laço de repetição, basta dividir o conteúdo do acumulador por cinco, e, com isso, obteremos a média dos preços. O bloco que realiza essa tarefa é demonstrado a seguir:

```
soma <- 0
para i de 1 ate 5 faca
    escreva("Preço produto: ")
    leia(preco)
    soma <- soma + preco
fimpara
escreva("Média preço: ", (soma/5))
```



Agora, vamos alterar o enunciado desse problema: como elaborar um algoritmo que calcule a média de preço entre cinco produtos fornecidos pelo usuário? Ao final, ele deve escrever na tela quais preços foram maiores que a média.

Veja que precisaremos guardar o valor de cada um dos preços, pois somente dessa maneira podemos determinar quais preços estão acima da média. Uma possível solução é apresentada no Código 1. Veja:

Código 1

```

{leitura das informações}
escreva("Preço produto: ")
leia(p1)
escreva("Preço produto: ")
leia(p2)
escreva("Preço produto: ")
leia(p3)
escreva("Preço produto: ")
leia(p4)
escreva("Preço produto: ")
leia(p5)
{cálculo da média dos preços}
media <- (p1+p2+p3+p4+p5)/5
escreva("Média preço: ", media)
{preços acima da média}
se (p1 > media ) entao
    escreva("Preço acima da média: ", p1)
fimse
se (p2 > media) entao
    escreva("Preço acima da média: ", p2)
fimse
se (p3 > media) entao
    escreva("Preço acima da média: ", p3)
fimse
se (p4 > media) entao
    escreva("Preço acima da média: ", p4)
fimse
se (p5 > media) entao
    escreva("Preço acima da média: ", p5)
fimse

```



Note que o trecho do algoritmo apresentado no Código 1 resolve corretamente o problema proposto, entretanto, o principal problema desse algoritmo é a eficiência.

Imagine, por exemplo, que, se em vez de calcular a média (e apresentar os preços acima dela) de cinco produtos, você precisasse realizar a mesma tarefa, mas com 10.000 produtos. Quantas linhas seriam necessárias para escrever esse algoritmo? Certamente, você precisaria de centenas de linhas.

Evoluindo nos estudos de estruturas de repetição, as estruturas de dados estabelecem mecanismos para a organização dos dados, de forma que podemos trabalhar com as informações de maneira prática e eficiente. Além disso, elas são capazes de manipular grandes quantidades de informações.

As estruturas de dados também são conhecidas como estruturas de dados homogêneas.

Observe, a seguir, a descrição de cada uma dessas estruturas.

2. Estruturas de dados homogêneas

As estruturas de dados homogêneas são utilizadas na manipulação de dados do mesmo tipo. De maneira prática e simples, podemos entender que uma estrutura homogênea é uma sequência de variáveis do mesmo tipo.

Imagine, por exemplo, que você precisa manipular uma grande quantidade de notas de alunos. Em vez de declarar uma variável para armazenar cada nota, você pode criar uma estrutura de dados homogênea capaz de armazenar uma sequência de notas.

Atenção!

Há dois tipos de estruturas homogêneas:

- Vetores.
- Matrizes



Nos próximos subtópicos, você conhecerá em detalhes os vetores e as matrizes.

Vetores

O vetor, também conhecido como *array*, pode ser definido como um conjunto de elementos do mesmo tipo armazenado sequencialmente na memória.

Para entender melhor esse conceito, voltaremos ao problema dos preços visto no início desta unidade. Se utilizarmos cinco variáveis para armazenar os preços, elas estarão distribuídas em posições aleatórias da memória. Entretanto, se utilizarmos um vetor de preços contendo cinco posições, cada posição desse vetor será definida sequencialmente na memória.

Na Figura 1, você pode notar a diferença entre a declaração de cinco variáveis

distintas para armazenamento dos preços e a declaração de um vetor. Veja que as variáveis estão distribuídas aleatoriamente na memória (p1, p2, p3, p4 e p5), enquanto o vetor organiza os dados sequencialmente.

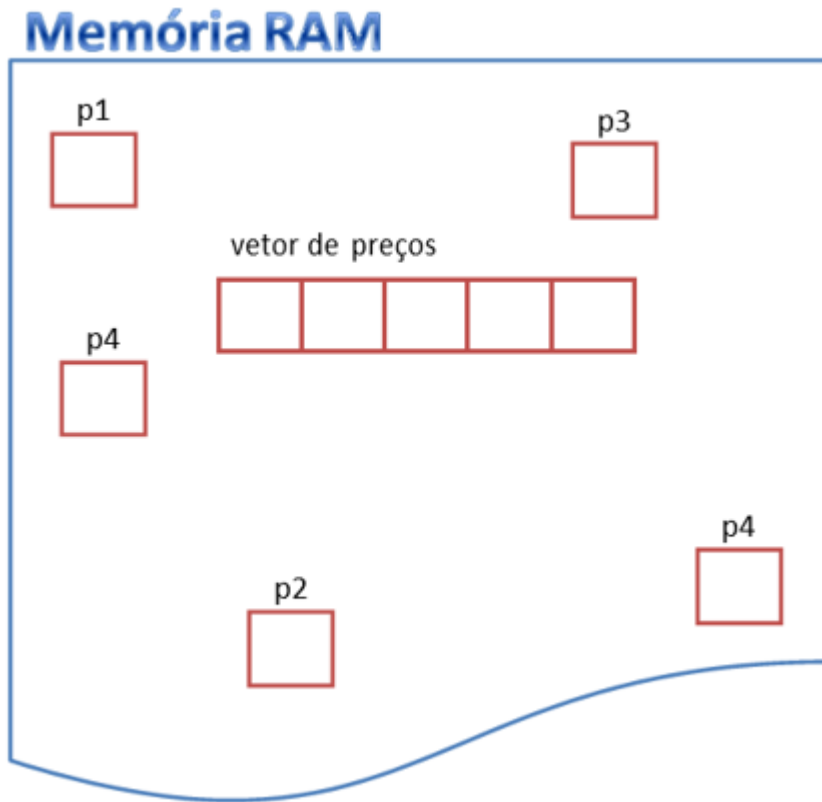


Figura 1 Organização dos dados na memória RAM.

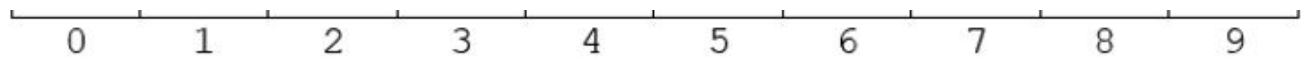
Como os vetores organizam as informações do mesmo tipo sequencialmente na memória, precisamos de um mecanismo para armazenar ou recuperar as informações de cada posição. Para isso, são utilizados valores inteiros, conhecidos como **índices**.

Um índice é um valor inteiro, **maior** ou **igual a zero**, utilizado para fazer referência a uma determinada posição do vetor. Vejamos um exemplo:

Considere um vetor que armazene as notas de dez alunos de uma instituição de ensino superior. Esse vetor e seus respectivos índices podem ser representados da seguinte maneira:

Vetor_de_notas



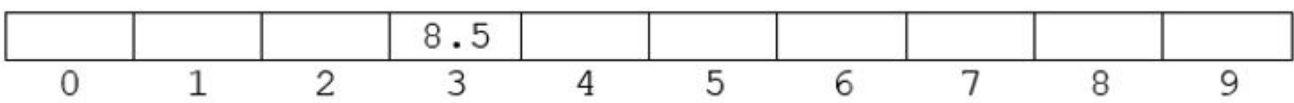


Os valores de zero até nove representam os índices que podem ser utilizados para armazenar ou recuperar informações no vetor. Por exemplo:

```
vetor_de_notas[3] <- 8.5
```

Armazenaremos o valor 8,5 na Posição 3 do `vetor_de_notas`; assim, o vetor terá a seguinte representação:

Vetor_de_notas



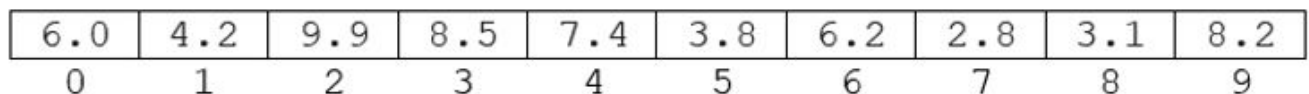
Desse modo, podemos utilizar o modelo definido anteriormente para armazenar valores nas demais posições do vetor:

```
vetor_de_notas[0] <- 6.0  
vetor_de_notas[1] <- 4.2  
vetor_de_notas[2] <- 9.9  
...  
vetor_de_notas[8] <- 3.1  
vetor_de_notas[9] <- 8.2
```



O resultado é um vetor com as seguintes informações:

Vetor_de_notas



Após analisar os vetores anteriores, podemos perceber que um vetor é uma estrutura de dados homogênea **unidimensional**, pois contém uma única linha e várias colunas. O número de colunas determina a quantidade de informações que serão armazenadas pelo vetor. Por exemplo: um vetor de inteiros contendo 1000 colunas será capaz de armazenar 1000 variáveis do tipo inteiro.

Da mesma forma que as variáveis, os vetores também precisam ser declarados. A declaração de um vetor em linguagem algorítmica tem a seguinte sintaxe:

```
<nome_do_vetor>: vetor [<indice_inicial>..indice_final] de <tipo_de_dados>
```

Para definir o nome do vetor **<nome_do_vetor>**, você deverá seguir os mesmos modelos de definição de nomes para variáveis. Assim, os parâmetros **<índice_inicial>** e **<índice_final>** determinam o número de posições do vetor, e o parâmetro **<tipo_de_dados>** especifica qual será o tipo de dado armazenado pelo vetor.

Atenção!

Lembre-se de que vetores **sempre** armazenam dados do mesmo tipo.

Observe, a seguir, alguns exemplos de declarações de vetores:

```
var
{vetor capaz de armazenar 12 números inteiros}
vet_inteiros: vetor [0..11] de inteiro
{vetor capaz de armazenar 50 números reais}
vet_numeros: vetor [0..49] de real
{vetor capaz de armazenar 10 caracteres}
vet_caracteres: vetor [0..9] de caractere
```



Você conseguiu compreender a execução dos comandos de vetores? Para testar se de fato aprendeu, responda à questão a seguir.

Agora, observe que, para ilustrar a utilização de vetores, criaremos um algoritmo que armazene, por meio de um vetor, cinco valores inteiros informados pelo usuário. A leitura dos dados será realizada por uma estrutura de repetição, e, ao final, os valores recebidos serão exibidos na tela.

Código 2

```

algoritmo "vetores"
var
    i : inteiro
    vet_inteiros: vetor [0..4] de inteiro
inicio
    para i de 0 ate 4 faca
        escreva("Informe um número: ")
        leia(vet_inteiros[i])
    fimpara
    para i de 0 ate 4 faca
        escreval("Posição ", i, " = ", vet_inteiros[i])
    fimpara
fimalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```

Informe um número: 10
Informe um número: 20
Informe um número: 30
Informe um número: 40
Informe um número: 50

Posição 0 = 10
Posição 1 = 20
Posição 2 = 30
Posição 3 = 40
Posição 4 = 50

```



Agora, resolveremos o problema proposto no início desta unidade. Para isso, elaboraremos um algoritmo que seja capaz de calcular a média de preço entre cinco produtos fornecidos pelo usuário e que, ao final, exiba os preços maiores que a média.

Código 3

```

algoritmo "precos_acima_da_media"
var
    i : inteiro
    soma, media: real
    vet_precos: vetor [0..4] de real
inicio
    para i de 0 ate 4 faca
        escreva("Informe o preço: ")
        leia(vet_precos[i])
        soma <- soma + vet_precos[i]
    fimpara
    media <- soma / 5
    escreval("Média dos preços: ", media)
    para i de 0 ate 4 faca
        se (vet_precos[i] > media) entao
            escreval("Preço acima da média: ", vet_precos[i])
        fimse
    fimpara
fimalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:



```

Informe o preço: 5,5
Informe o preço: 2,4
Informe o preço: 9,8
Informe o preço: 10,5
Informe o preço: 7,2
Média dos preços: 7,08
Preço acima da média: 9.8
Preço acima da média: 10.5
Preço acima da média: 7.2

```

A busca de informações em estruturas do tipo vetores é importante para a construção dos algoritmos. A maneira mais simples de realizar uma busca é percorrer o vetor do início ao fim. Essa abordagem é conhecida como **busca sequencial**. O exemplo, no Código 4, demonstra como realizar a busca de um determinado elemento no vetor. O algoritmo varre o vetor sequencialmente e, caso encontre o elemento desejado, exibe sua respectiva posição. Observe:

Código 4

```

algoritmo "busca_vetor"
var
    i, numero, posicao: inteiro
    vet_numeros: vetor [0..4] de inteiro
inicio
    para i de 0 ate 4 faca
        escreva("Informe um número: ")
        leia( vet_numeros[i] )
    fimpara
    escreva("Qual número deseja pesquisar? ")
    leia(numero)
    posicao <- -1
    para i de 0 ate 4 faca
        se ( vet_numeros[i] = numero ) entao
            posicao <- i
        fimse
    fimpara
    se ( posicao = -1 ) entao
        escreva("O número pesquisado não foi encontrado.")
    senao
        escreva("O número foi encontrado na posição ", posicao)
    fimse
fimalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução a seguir. As informações utilizadas como entrada são apenas ilustrativas. 

Resultado da execução do algoritmo:

```

Informe um número: 15
Informe um número: 22
Informe um número: 8
Informe um número: 16
Informe um número: 4
Qual número deseja pesquisar? 16
O número foi encontrado na posição 3

```

No Código 5, descrito a seguir, elaboraremos um algoritmo que determina qual é o maior e qual é o menor elemento armazenado no vetor. Além disso, informaremos ao usuário qual é a posição desses elementos. O vetor utilizado deverá ter seis posições e conter elementos do tipo real.

Código 5

```

algoritmo "maior_menor"
var
    i : inteiro
    vet_elementos: vetor [0..5] de real
    maior_elem, menor_elem: real
    maior_pos, menor_pos: inteiro
inicio
    para i de 0 ate 5 faca
        escreva("Elemento: ")
        leia( vet_elementos[i] )
    fimpara
    maior_elem <- vet_elementos[0]
    maior_pos <- 0
    menor_elem <- vet_elementos[0]
    maior_pos <- 0
    para i de 1 ate 5 faca
        se ( vet_elementos[i] > maior_elem ) entao
            maior_elem <- vet_elementos[i]
            maior_pos <- i
        fimse
        se ( vet_elementos[i] < menor_elem ) entao
            menor_elem <- vet_elementos[i]
            menor_pos <- i
        fimse
    fimpara
    escreval("Maior elemento:", maior_elem, " posição:", maior_pos)
    escreval("Menor elemento:", menor_elem, " posição:", menor_pos)
finalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```

Elemento: 1
Elemento: 2
Elemento: 3
Elemento: 4
Elemento: 5
Elemento: 6
0 maior elemento é 6 encontrado na posição 5
0 menor elemento é 1 encontrado na posição 0

```

Uma questão importante quando trabalhamos com vetores é a necessidade de ordenar os elementos armazenados. Como você poderá observar, o Código 6 demonstra como é possível ordenar facilmente os elementos de um vetor.

Nesse algoritmo, dez valores são fornecidos pelo usuário, os quais serão ordenados e exibidos na tela.

Código 6

```
algoritmo "ordenar_vetor"
var
    vet_numeros: vetor [0..9] de inteiro
    i,j, aux: inteiro
inicio
    i <- 0
    repita
        escreva("Informe um número: ")
        leia(vet_numeros[i])
        i <- i + 1
    ate ( i = 10 )
    para i de 0 ate 9 faca
        para j de 0 ate 8 faca
            se (vet_numeros[j] > vet_numeros[j+1]) entao
                aux <- vet_numeros[j]
                vet_numeros[j] <- vet_numeros[j+1]
                vet_numeros[j+1] <- aux
            fimse
        fimpara
    fimpara
    para i de 0 ate 9 faca
        escreval(vet_numeros[i])
    fimpara
fimalgoritmo
```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:


```
Informe um número: 9
Informe um número: 2
Informe um número: 4
Informe um número: 8
Informe um número: 3
Informe um número: 6
Informe um número: 1
Informe um número: 5
Informe um número: 10
Informe um número: 7
1
2
3
4
5
6
7
8
9
10
```

Faça uma pausa em seus estudos e avalie sua aprendizagem em relação aos conteúdos apresentados anteriormente. Para isso, responda à questão proposta a seguir.



Agora que compreendeu os vetores e identificou como executá-los, vamos entender como é a execução das matrizes na linguagem de programação.

Matrizes

As matrizes são outro tipo de estrutura de dados homogênea e permitem a manipulação de informações em muitas dimensões.

Como vimos anteriormente, os vetores manipulam dados em uma única dimensão e, por isso, são conhecidos como estruturas de dados unidimensionais. Já as matrizes permitem a manipulação de informações em muitas dimensões e, por essa razão, são conhecidas como estruturas multidimensionais.

Desse modo, a matriz mais simples é chamada de matriz bidimensional, e, nela, os dados são representados em duas dimensões.

Entretanto, nas matrizes bidimensionais, os dados são dispostos em linhas e colunas, como em uma tabela, e o acesso às informações é realizado por meio de dois índices: um índice para representar a linha e outro para representar a coluna.

Uma matriz bidimensional de cinco linhas e seis colunas é representada graficamente a seguir, conforme Figura 2:

		colunas					
		0	1	2	3	4	5
linhas	0						
	1						
	2						
	3						
	4						



Fonte: desenvolvida pelos autores.

Figura 2 Estrutura de uma matriz bidimensional.

Para atribuir o valor 100 à linha 3 e coluna 4 da Tabela, você deve utilizar o seguinte comando:

```
matriz[3,4] <- 100
```

Como resultado, temos:

		colunas					
		0	1	2	3	4	5
linhas	0						
	1						

1						
2						
3					100	
4						

Fonte: desenvolvida pelos autores.

Figura 3 Estrutura de uma matriz bidimensional contendo o valor 100 na posição (3,4).

Assim, para acessar qualquer posição de uma matriz, basta utilizar um valor que represente o número da linha e outro que represente o número da coluna. De forma geral, temos:

```
<nome_da_matriz>[numero_da_linha, numero_da_coluna] <- valor_desejado
```

Também podemos utilizar o modelo definido anteriormente para armazenar valores nas demais posições da matriz:

```
matriz[0,0] <- 120
matriz[0,1] <- 180
matriz[0,2] <- 90
matriz[0,3] <- 30
matriz[0,4] <- 220
matriz[0,5] <- 10
matriz[1,0] <- 440
...
matriz[4,5] <- 80
```



		colunas					
		0	1	2	3	4	5
linhas	0	120	180	90	30	220	10
	1	440	12	43	87	43	249
	2	46	123	32	34	487	143
	3	85	76	378	68	100	93
	4						

4	48	92	66	23	82	80
---	----	----	----	----	----	----

Fonte: desenvolvida pelos autores.

Figura 4 Estrutura de uma matriz bidimensional contendo todos os elementos preenchidos.

Observe que a estrutura de uma matriz é muito semelhante à estrutura de um vetor. A principal diferença é a utilização de mais um índice para representar as linhas. Dessa forma, a declaração também é bastante semelhante.

nome_da_matriz: vetor [<lin_inicial>..<lin_final>, <col_inicial>..<col_final>] de <tipo_de_dados>

Índices para as linhas Índices para as colunas

Fonte: desenvolvida pelos autores.

Figura 5 Diferença entre a estrutura de uma matriz e um vetor.

Para declararmos uma matriz, precisamos informar o número de linhas e o número de colunas que ela terá; assim, a declaração tem a sintaxe apresentada na Figura 5.

Verifique, a seguir, alguns exemplos de declaração de matrizes:

```
{matriz contendo 10 linhas e 10 colunas para armazenar reais}

mat_numeros: vetor [0..9, 0..9] de real

{matriz contendo 2 linhas e 8 colunas para armazenar inteiros}

mat_inteiros: vetor [0..1, 0..7] de inteiro
```

A capacidade de elementos de uma matriz pode ser obtida por meio da multiplicação do número de linhas pelo número de colunas. Por exemplo: uma matriz de três linhas e cinco colunas é capaz de armazenar 15 elementos, ou seja, **(três linhas * cinco colunas) = 15 elementos**.

Para ilustrar a utilização de matrizes, escreveremos um algoritmo que declara uma matriz de duas linhas e três colunas. Em seguida, vamos atribuir valores

informados pelo usuário a cada posição da matriz e, finalmente, exibir os valores recebidos na tela. Veja:

Código 7

```
algoritmo "matrizes"
var
    mat_numeros:vetor[0..1, 0..2] de inteiro
    lin: inteiro
    col: inteiro
inicio
    para lin de 0 ate 1 faca
        para col de 0 ate 2 faca
            escreva("Informe um número: ")
            leia(mat_numeros[lin,col])
        fimpara
    fimpara
    para lin de 0 ate 1 faca
        para col de 0 ate 2 faca
            escreval("Linha=",lin," Coluna=",col," Número= ",mat_numeros[lin,col])
        fimpara
    fimpara
fimalgoritmo
```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```
Informe um número: 1
Informe um número: 2
Informe um número: 3
Informe um número: 4
Informe um número: 5
Informe um número: 6
Linha= 0 Coluna= 0 Número= 1
Linha= 0 Coluna= 1 Número= 2
Linha= 0 Coluna= 2 Número= 3
Linha= 1 Coluna= 0 Número= 4
Linha= 1 Coluna= 1 Número= 5
Linha= 1 Coluna= 2 Número= 6
```

Graficamente, a matriz do algoritmo presente no Código 7 pode ser representada da seguinte forma:

		colunas		
		0	1	2
linhas	0	1	2	3
	1	4	5	6

Fonte: desenvolvida pelos autores.

Figura 5 Representação gráfica da execução do algoritmo.

No próximo exemplo, utilizaremos uma matriz de três linhas e três colunas do tipo real. O objetivo desse algoritmo é receber os dados e armazená-los em cada posição da matriz. Em seguida, o algoritmo deverá retornar a soma de uma determinada linha informada pelo usuário. Considere, por exemplo, a matriz:

		colunas		
		0	1	2
linhas	0	5	10	15
	1	20	25	30
	2	35	40	45



Fonte: desenvolvida pelos autores.

Figura 6 Exemplo de uma possível matriz de entrada para o algoritmo.

A soma dos elementos da linha 0 da matriz é igual a 30, a soma dos elementos da linha 1 é igual a 75, e a soma dos elementos da linha 2 é igual a 120.

No Código 8, a seguir, o algoritmo demonstra a soma dos elementos de uma determinada linha da matriz. Observe:

Código 8

```

algoritmo "matrizes"
var
    mat_numeros:vetor[0..2, 0..2] de real
    lin: inteiro
    col: inteiro
    linha: inteiro
    soma : real
Inicio
    para lin de 0 ate 2 faca
        para col de 0 ate 2 faca
            escreva("lin: ",lin," Col:", col," Informe um número: ")
            leia(mat_numeros[lin,col])
        fimpara
    fimpara
    escreva("Informe uma linha da matriz: ")
    leia(linha)
    soma <- 0
    para lin de 0 ate 2 faca
        para col de 0 ate 2 faca
            se ( lin = linha ) entao
                soma <- soma + mat_numeros[lin,col]
            fimse
        fimpara
    fimpara
    escreval("Soma dos elementos da linha ", linha," =", soma)
finalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```

lin: 0 col: 0 Informe um número: 1
lin: 0 col: 1 Informe um número: 2
lin: 0 col: 2 Informe um número: 3
lin: 1 col: 0 Informe um número: 4
lin: 1 col: 1 Informe um número: 5
lin: 1 col: 2 Informe um número: 6
lin: 2 col: 0 Informe um número: 7
lin: 2 col: 1 Informe um número: 8
lin: 2 col: 2 Informe um número: 9
Informe uma linha da matriz: 1
Soma dos elementos da linha 1 = 15

```

A seguir, apresentaremos um algoritmo que realiza o controle de notas dos alunos de uma universidade. Para isso, será utilizada uma estrutura do tipo

matriz. O número de linhas da matriz corresponde ao número de alunos, por exemplo. Para facilitar a explicação, utilizaremos apenas cinco linhas.

A matriz terá três colunas. Na primeira coluna, será armazenada a nota da 1ª prova; na segunda, a nota da 2ª prova; e, finalmente, na terceira, a média das notas.

Inicialmente, o algoritmo deverá receber as notas das provas e calcular a média. Por fim, todos os dados deverão ser exibidos na tela. Observe o Código 9:

Código 9

```
algoritmo "matrizes"
var
    notas:vetor[0..4, 0..2] de real
    lin: inteiro
    col: inteiro
inicio
    para lin de 0 ate 4 faca
        escreva("Nota 1ª Prova: ")
        leia(notas[lin,0])
        escreva("Nota 2ª Prova: ")
        leia(notas[lin,1])
        notas[lin,2] <- (notas[lin,0]+notas[lin,1])/2
    fimpara
    escreval("1ª Prova, 2ª Prova, Média")
    lin <- 0
    enquanto ( lin <= 4 ) faca
        escreval(notas[lin,0], ", ", notas[lin,1], ", ", notas[lin,2])
        lin <- lin+1
    fimenquanto
fimalgoritmo
```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

Nota 1ª Prova: 8
Nota 2ª Prova: 6
Nota 1ª Prova: 4,5
Nota 2ª Prova: 9
Nota 1ª Prova: 3,2
Nota 2ª Prova: 8
Nota 1ª Prova: 9,4
Nota 2ª Prova: 5,6
Nota 1ª Prova: 2,3
Nota 2ª Prova: 6,4

1ª Prova,	2ª Prova,	Média
8,	6,	7
4,5,	9,	6,75
3,2,	8,	5,6
9,4,	5,6,	7,5
2,3,	6,4,	4,35

Para encerrarmos esta unidade, novamente propomos a você um momento de reflexão sobre o assunto abordado. E, para conferir se você conseguiu entender bem o que foi apresentado, responda à questão a seguir:

Estamos encerrando os estudos desta unidade, todavia, antes disso, procure ampliar ainda mais seu conhecimento. Para isso, realize as leituras indicadas a seguir.



Que tal ampliar sua compreensão sobre vetores, matrizes e estruturas?

Para isso, faça a leitura dos seguintes capítulos:

- Capítulo 6 – *Vetores, Matrizes e Estruturas*, da obra *Linguagem e lógica de programação* (ALVEZ, 2014).
- Capítulo 7 – *Aplicações Básicas com Matrizes de uma Dimensão* e Capítulo 8 – *Estruturas de Dados Homogêneas de Duas Dimensões*, da obra: *Algoritmos – Lógica para Desenvolvimento de Programação de Computadores* (MANZANO; OLIVEIRA, 2019). Estas obras estão disponíveis na Minha Biblioteca.

3. Considerações

Nesta unidade, além de ter a oportunidade de aprender as principais estruturas de dados, você também pôde estudar como é possível declarar e manipular vetores e matrizes. Pôde perceber, ainda, que essas estruturas permitem o armazenamento de grandes quantidades de dados.

Na próxima unidade, observaremos a construção de algoritmos utilizando os conceitos de programação modular.

Até mais!



Unidade 5 – Programação Modular

Objetivos

- Construir algoritmos utilizando os conceitos de programação modular.
- Reconhecer e construir sub-rotinas recursivas.

Conteúdos

- **Programação modular.**
- **Variáveis** globais e variáveis locais.
- Procedimentos.
- Funções.
- Recursividade.

Problematização



Como utilizar programação modular? Como inserir procedimentos em um algoritmo? O que é função? Como implementar funções em um algoritmo?

Orientações para o estudo

Esta última unidade está estruturada para fornecer a base necessária para compreender a programação modular, por isso, faça uma leitura atenta dos capítulos que indicamos para ampliação de seus estudos sobre esse assunto. Vale destacar também que os estudos da unidade e as questões que deverá responder serão essenciais para sua autoavaliação, pois este será um bom momento para consolidar e aplicar os conhecimentos em sua prática profissional e acadêmica.

Vamos lá! Bons estudos!

1. Introdução

No decorrer do estudo das unidades anteriores, você teve a oportunidade de aprender diversos conceitos a respeito da construção de algoritmos. Portanto, neste momento, você já é capaz de desenvolver inúmeros tipos de algoritmos para os mais variados propósitos.

Dessa forma, podemos perceber que a complexidade do desenvolvimento dos algoritmos aumenta, e, assim, dois problemas tornam-se evidentes:

1. O aumento significativo do número de linhas.
2. A dificuldade para a organização do código.

Atenção!

Para solucionar tais problemas, nesta unidade, aprenderemos uma técnica muito interessante para a reutilização e organização dos blocos de comandos presentes nos algoritmos. Essa técnica é conhecida como "programação modular".

O objetivo da **programação modular** é dividir o algoritmo em pequenas partes, ou módulos, que são chamadas de "**sub-rotinas**".



Ao utilizar as sub-rotinas, temos a vantagem de reaproveitar partes do código ao longo de todo o algoritmo; com isso, não precisamos reescrever trechos do código. Como resultado, teremos um algoritmo mais organizado e com um número menor de linhas.

Também vale ressaltar outro aspecto importante das sub-rotinas – é por meio delas que dividimos o problema da construção do algoritmo em pequenas partes, tornando-o mais fácil de ser resolvido. Essa abordagem para a resolução de problemas é conhecida como "dividir-para-conquistar".

Conforme demonstrado no trecho de código a seguir, o cálculo da média entre dois valores é realizado diversas vezes. Veja:

```

escreva("Informe um número: ")
leia(num1)
escreva("Informe um número: ")
leia(num2)
soma <- num1+num2
media <- (soma/2)
escreva("Média = ", media)

escreva("Informe um número: ")
leia(num1)
escreva("Informe um número: ")
leia(num2)
soma <- num1+num2
media <- (soma/2)
escreva("Média = ", media)

escreva("Informe um número: ")
leia(num1)
escreva("Informe um número: ")
leia(num2)
soma <- num1+num2
media <- (soma/2)
escreva("Média = ", media)

```

Note que a sequência lógica realiza várias vezes o mesmo cálculo.



Veja que podemos melhorar esse problema de lógica utilizando os conceitos da programação modular. A ideia é criar um módulo, ou sub-rotina, capaz de realizar o cálculo da média. Essa sub-rotina recebe dois valores, calcula a média e devolve o resultado.

De modo geral, a sub-rotina para o cálculo da média poderia ser implementada da seguinte forma:

```

subrotina calculo_media(num1 real, num2 real)

início

    soma <- num1+num2

    retorne (soma/2)

fimsubrotina

```

Analisando o algoritmo anterior, verificamos que, quando definimos uma sub-rotina, precisamos especificar um nome. Nesse caso, o nome escolhido foi **cal-**

culo_media. Temos, também, um conjunto de variáveis que pode ser passado para sub-rotinas, chamado de **parâmetros**.

Para a sub-rotina **calcula_media**, dois parâmetros foram definidos: **num1**, do tipo real, e **num2**, também do tipo real. Esses parâmetros serão utilizados para que a sub-rotina receba os valores necessários para o cálculo da média.

Na Linha "**soma <- num1+num2**", é calculada a soma entre os valores fornecidos no parâmetro. Finalmente, na Linha "**retorne (soma/2)**", a média é calculada, e o resultado encontrado é devolvido para o programa principal.

Alterando o trecho de código exibido inicialmente, podemos visualizar como ocorre a utilização de sub-rotinas na construção dos algoritmos. Verifique:

```
escreva("Informe um número: ")
leia(num1)
escreva("Informe um número: ")
leia(num2)
escreva("Média = ", calcula_media(num1,num2))

escreva("Informe um número: ")
leia(num1)
escreva("Informe um número: ")
leia(num2)
escreva("Média = ", calcula_media(num1,num2))

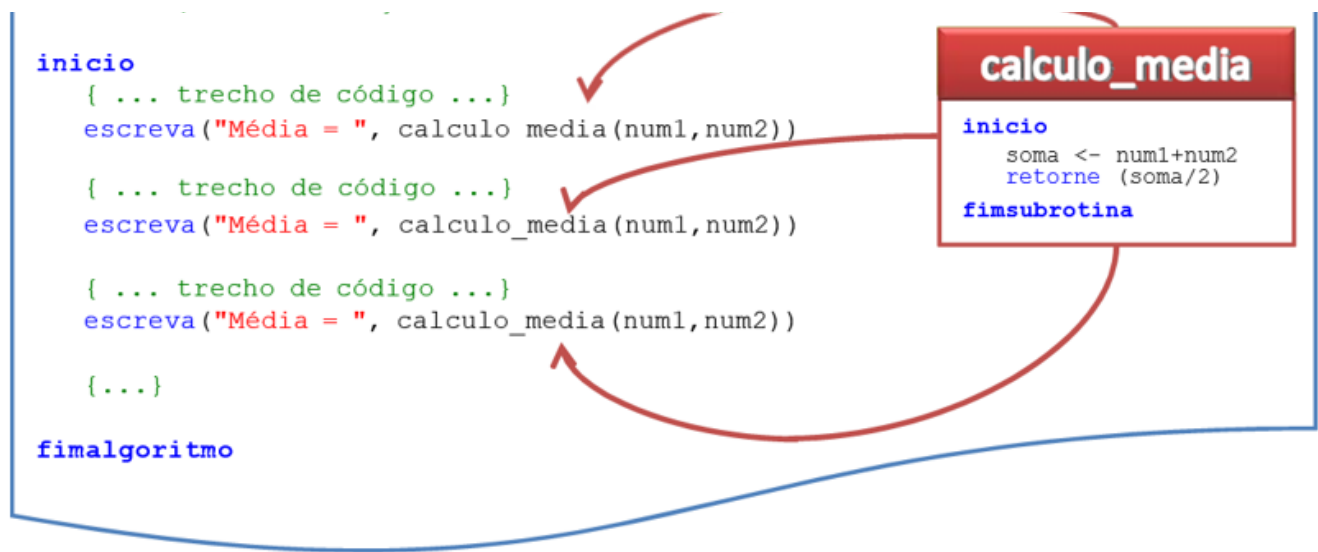
escreva("Informe um número: ")
leia(num1)
escreva("Informe um número: ")
leia(num2)
escreva("Média = ", calcula_media(num1,num2))
```



No trecho do código anterior, as linhas em destaque representam as chamadas para a sub-rotina **calcula_media**. Note que são passados dois parâmetros para sub-rotina, e, como resultado, é obtida a média entre os números (num1 e num2).

Visualmente, esse algoritmo pode ser representado da seguinte forma:

```
algoritmo "CalculoMedia"
declare
    { ... declaração das variáveis ... }
```



Fonte: desenvolvido pelos autores.

Figura 1 **Algoritmo** – cálculo da média.

As técnicas de modularização de programas permitem a definição de dois tipos de sub-rotinas, que são conhecidos como:

- Procedimentos.
- Funções.



No próximo tópico, veremos a descrição dessas sub-rotinas.

2. Procedimentos

Os procedimentos são a maneira mais simples de criação de sub-rotinas. Um procedimento é um módulo dentro do algoritmo utilizado para executar um conjunto de instruções.

Atenção!

A principal característica de um procedimento é que ele executa instruções. Porém, não retorna nenhum tipo de resultado para o programa principal.

Observe, a seguir, a sintaxe para a declaração de um procedimento:

```
procedimento <nome_do_procedimento> (<lista_de_parametros>)
```

```
inicio
```

```
    {bloco de instruções}
```

```
fimprocedimento
```

Analisando a sintaxe anterior, observe que a definição de um **procedimento** sempre deve começar com a palavra reservada **procedimento**. Em seguida, é necessário especificar o seu nome, o qual será utilizado ao longo do algoritmo para a execução do procedimento. Além disso, ele pode conter uma lista de parâmetros, que são variáveis passadas para o procedimento visando a algum tipo de cálculo.

Os procedimentos possuem um bloco delimitador, cuja abertura é especificada com a palavra **início**, e o fim é especificado com a palavra reservada **fimprocedimento**.

Para ilustrar a criação de procedimentos dentro da estrutura de um algoritmo, elaboraremos um algoritmo capaz de exibir um nome na tela por um determinado número de vezes. As informações a respeito do nome e do número de vezes que ele será exibido deverão ser especificadas pelo usuário. Observe o Código 1:



Código 1


```

algoritmo "exemplo_procedimento"
var
    nome: caractere
    numero: inteiro
procedimento exibirnome(nome: caractere; numero: inteiro)
var
    i : inteiro
inicio
    para i de 1 ate numero faca
        escreva(nome)
    fimpara
fimprocedimento
inicio
    escreva("Nome: ")
    leia(nome)
    escreva("Número de vezes: ")
    leia(numero)
    exibirnome(nome,numero)
fimalgoritmo

```

Neste código, as Linhas de **procedimento** a **fimprocedimento** especificam o procedimento *exibirnome*. Note que esse procedimento recebe como **parâmetro** o **nome** e o número de vezes que ele deve ser repetido. Verifique, também, que os parâmetros passados para o procedimento são separados por ponto e vírgula.



Outro aspecto interessante é a declaração da variável **i** dentro do procedimento. As sub-rotinas (procedimentos e funções) permitem a declaração de variáveis no seu interior, chamadas de "**variáveis locais**". Em contrapartida, variáveis declaradas no cabeçalho principal do algoritmo são conhecidas como **variáveis globais**.

A Linha "**exibirnome(nome,numero)**" demonstra como é possível realizar a chamada do procedimento dentro da estrutura principal do algoritmo.

Vejamos um exemplo de execução:

Resultado da execução do algoritmo:

```
Nome: João da Silva
Número de vezes: 2
João da Silva
João da Silva
```

Agora, realizaremos uma alteração no algoritmo, de forma que o procedimento **exibirnome** seja chamado duas vezes. Assim, temos os seguintes comandos:

```
exibirnome(nome, numero)

exibirnome(nome, numero)
```

Resultado da execução do algoritmo:

```
Nome: João da Silva
Número de vezes: 2
João da Silva
João da Silva
João da Silva
João da Silva
```



Observe que, como o procedimento **exibirnome** é chamado duas vezes, o resultado será a exibição do nome quatro vezes na tela. Isso ocorre porque, em cada chamada do procedimento, o nome é exibido duas vezes.

Como já comentamos, quando trabalhamos com a programação modular, podemos declarar variáveis de duas maneiras:

1. Variáveis globais.
2. Variáveis locais.

A diferença entre variáveis globais e variáveis locais é simples. Uma **variável global** pode ser utilizada em qualquer parte do algoritmo, enquanto uma **variável local** é utilizada apenas no interior da sub-rotina em que ela foi declarada.

Vejamos um exemplo:

Código 2

```

algoritmo "variaveis_globais"
var
    nome: caractere
procedimento exhibirnome()
inicio
    escreval(nome)
fimprocedimento
inicio
    nome <- "João da Silva"
    escreval("Nome: ", nome)
    exhibirnome()
finalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir. As informações utilizadas como entrada são apenas ilustrativas.

Resultado da execução do algoritmo:

```

Nome: João da Silva
João da Silva

```

Verifique que, na Linha “**nome: caractere**” do algoritmo constante no Código 3, temos a **declaração da variável global** nome.



Atenção!

Lembre-se de que as variáveis globais são declaradas no cabeçalho do algoritmo logo após a palavra reservada **declare**. Não confunda variável global com variável criada dentro das declarações da sub-rotina.

Como **nome** é uma variável global, podemos utilizá-la dentro do bloco principal do algoritmo, como demonstrado nas Linhas **nome <- "João da Silva"** e **escreval("Nome: ", nome)**. Além disso, podemos utilizar a variável nome dentro dos procedimentos, como pode ser visualizado na Linha “**escreval(nome)**”.

Vejamos outro exemplo, só que agora com a variável nome sendo declarada como local:

Código 3

```
algoritmo "variáveis_locais"
procedimento exhibirnome()
var
    nome: caractere
inicio
    nome <- "João da Silva"
    escreva(nome)
fimprocedimento
inicio
    exhibirnome()
fimalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:

João da Silva

É importante observar que, se tentarmos utilizar uma variável local dentro do bloco principal do algoritmo, teremos um **erro de execução**. Isso torna inviável o funcionamento do algoritmo. Por exemplo:



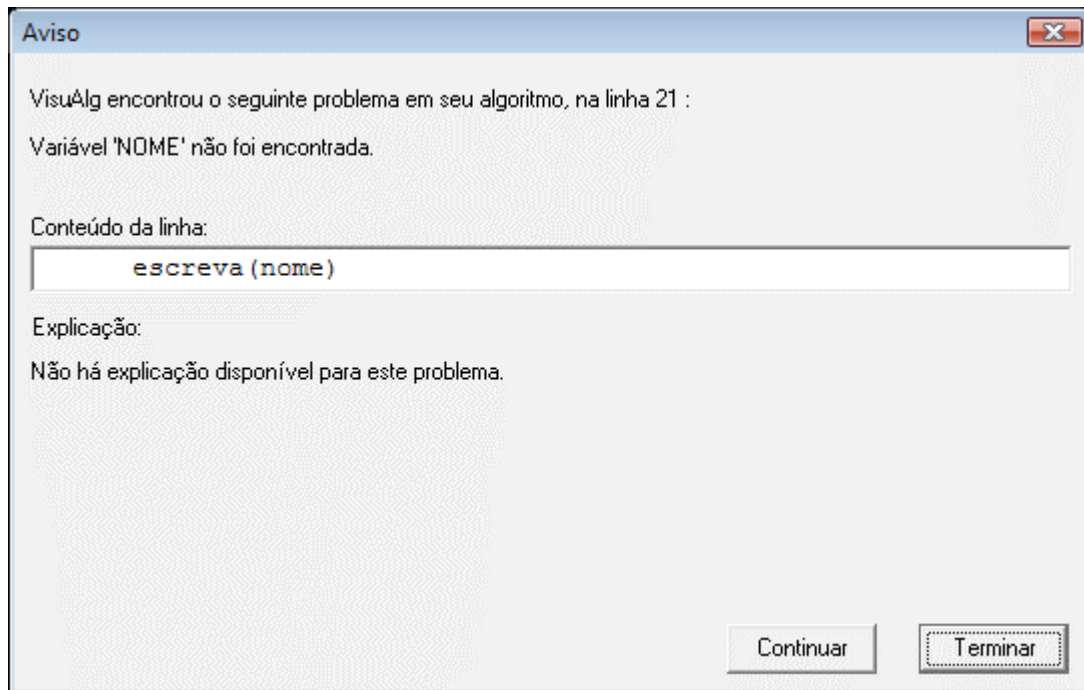
Código 4

```
algoritmo "variaveis_locais"
procedimento exhibirnome()
var
    nome: caractere
inicio
    nome <- "João da Silva"
    escreva(nome)
fimprocedimento
inicio
    escreva(nome)
    exhibirnome()
fimalgoritmo
```

Ao analisar o Código 4, podemos ver que a Linha “**escreva(nome)**” tenta exibir na tela o valor contido na variável **nome**. Essa instrução produz um erro de execução, **pois a variável nome é local** e, portanto, pode ser utilizada apenas

dentro do procedimento no qual ela foi definida.

Se você estiver utilizando o programa VisuAlg, receberá a mensagem a seguir (Figura 2):



Fonte: desenvolvida pelos autores.

Figura 2 Mensagem exibida no programa VisuAlg.

Outra característica interessante das sub-rotinas é a possibilidade de haver a **passagem de parâmetros**. Os parâmetros permitem que as variáveis do bloco principal do algoritmo sejam passadas para as sub-rotinas. Esse conceito pode ser aplicado aos procedimentos e às funções. As funções serão apresentadas no próximo tópico.

Quando passamos um parâmetro para uma sub-rotina, este tem a função de uma variável, a qual pode ser manipulada ao longo da sub-rotina. O exemplo a seguir demonstra a utilização do conceito de passagem de parâmetros:

Código 5

```

algoritmo "passagem_de_parametros"
var
    numero: inteiro
procedimento quadrado(valor: inteiro)
inicio
    escreva("Quadrado de ", valor, " = ", (valor*valor))
fimprocedimento
inicio
    numero <- 2
    quadrado(numero)
fimalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:

```

Quadrado de 2 = 4

```

Verifique que, no exemplo anterior, foi declarada uma variável do tipo inteiro, denominada **numero**. Essa variável recebe, no bloco principal do algoritmo, o valor **2**. Em seguida, é executada a chamada do procedimento **quadrado**, em que a variável **numero** é passada como parâmetro.

Note que, na definição do procedimento **quadrado**, a variável que a princípio era chamada de **nome** passa a ser chamada de **valor**. Desse modo, dentro do procedimento, a variável pode ser utilizada com o nome de valor.

No próximo exemplo, veremos o conceito em que os parâmetros são utilizados para passar o conteúdo de duas variáveis do bloco principal para o procedimento. Observe:

Código 6

```

algoritmo "passagem_de_parametros"
var
    num1, num2: real
procedimento dividir(a:real; b: real)
inicio
    escreval("Divisão = ", (a/b) )
fimprocedimento
inicio
    num1 <- 10
    num2 <- 2
    dividir(num1,num2)
    dividir(num2,num1)
finalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:

```

Divisão = 5
Divisão = 0,2

```



No algoritmo anterior, observe que as Linhas “**dividir(num1, num2)**” e “**dividir(num2, num1)**” realizam a chamada do procedimento dividir. É importante notar a ordem em que as variáveis **num1** e **num2** são passadas como parâmetros. Na chamada da Linha “**dividir(num1, num2)**”, temos **num1** e depois **num2**. Isso significa que o parâmetro **a** terá o valor da variável **num1**, e o parâmetro **b**, o valor da variável **num2**. Com isso, o resultado da divisão será **10/2 = 5**.

Na Linha de código “**dividir(num2, num1)**”, temos o processo inverso, pois, na chamada, temos **num2** e, em seguida, **num1**. Assim, o parâmetro **a** receberá o valor da variável **num2**, e o parâmetro **b**, o valor da variável **num1**. Como resultado, temos **2/10=0,2**.

Agora, vejamos outra utilidade da passagem de parâmetros.

A passagem de parâmetros para uma sub-rotina, seja procedimento, seja função, pode ser realizada de duas maneiras:

- Passagem de parâmetro por valor.
- Passagem de parâmetro por referência.

Na **passagem por valor**, qualquer tipo de alteração das variáveis utilizadas como parâmetro não é propagado no bloco principal do algoritmo. Isso significa que **podemos alterar o valor do parâmetro dentro da sub-rotina** e, quando a execução da sub-rotina terminar, o valor original (antes da chamada da sub-rotina) continua o mesmo.

A seguir, acompanhe um exemplo de passagem de parâmetros por valor:

Código 7

```
algoritmo "passagem_de_parametros_valor"
var
    numero : real
procedimento exibirvalor(valor: real)
inicio
    valor <- 20
    escreval("Número = ", valor)
fimprocedimento
inicio
    numero <- 10
    escreval("Número = ", numero)
    exibirvalor(numero)
    escreval("Número = ", numero)
finalgoritmo
```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:

```
Número = 10
Número = 20
Número = 10
```

O algoritmo do Código 7 possui uma variável global do tipo real, denominada **numero**. Inicialmente, no bloco principal do algoritmo, essa variável recebe o

valor **10** (Linha "**numero <- 10**"). Em seguida, na Linha "**escreval("Número = ", numero)**", o valor dessa variável é exibido na tela (Número = 10).

Na Linha "**exibirvalor(numero)**", é realizada uma chamada do procedimento **exibirvalor**, na qual a variável **numero** é passada como parâmetro. No interior do procedimento, a variável **numero**, que é referenciada pelo nome de valor, recebe o valor 20 (Linha "**valor <- 20**"). Em seguida, o conteúdo da variável é exibido na tela (Número = 20).

Veja que, na Linha "**valor <- 20**", o conteúdo da variável **valor** é alterado.

Entretanto, quando a execução do procedimento termina, o conteúdo da variável **numero** é novamente exibido na tela (Linha "**escreval("Número = ", numero)**"), abaixo da execução do procedimento. O resultado é o valor original da variável definido na Linha "**numero <- 10**".

Esse exemplo demonstrou a passagem de parâmetros por valor.



A principal característica dessa técnica é que qualquer alteração no conteúdo do parâmetro dentro da sub-rotina **não** é propagada para o bloco principal do algoritmo, ou seja, podemos alterar o conteúdo da variável no interior da sub-rotina e, quando a execução (da sub-rotina) terminar, esse valor será restaurado.

A outra forma de passagem de parâmetros é conhecida como **passagem de parâmetros por referência**. Nesse caso, as alterações ocorridas no interior da sub-rotina são propagadas para o bloco principal do algoritmo.

Para tornarmos essa característica possível, devemos indicar na definição da sub-rotina quais parâmetros deverão ser passados por referência. Para isso, basta utilizarmos a palavra reservada **var** antes do parâmetro.

Alteraremos o algoritmo presente no Código 7 para que a passagem de parâmetros seja realizada por referência. Acompanhe:

Código 8

```

algoritmo "passagem_de_parametros_referencia"
var
    numero : real
procedimento exibirvalor(var valor: real)
inicio
    valor <- 20
    escreval("Número = ", valor)
fimprocedimento
inicio
    numero <- 10
    escreval("Número = ", numero)
    exibirvalor(numero)
    escreval("Número = ", numero)
finalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:

```

Número = 10
Número = 20
Número = 20

```



Note que apenas uma alteração foi realizada na definição da sub-rotina, especificada na Linha “**procedimento exibirvalor(var valor: real)**”. Antes da definição do nome do parâmetro, é inserida a palavra reservada **var**.

Como resultado da execução, podemos notar que a alteração ocorrida no interior do procedimento (Linha “**valor <- 20**”) é refletida para o bloco principal do algoritmo. Assim, quando a execução do procedimento termina, a variável **numero** não contém seu valor original, pois foi alterada dentro do procedimento.

Uma vez que já aprendemos as variáveis globais e as variáveis locais, bem como a passagem de parâmetros por valor e por referência, estudaremos, então, um exemplo completo utilizando o procedimento.

Nesse exemplo, construiremos um algoritmo capaz de simular as principais funcionalidades de uma **calculadora**. Serão criados procedimentos para as principais operações: somar, subtrair, multiplicar e dividir. Além disso, criare-

mos um procedimento para calcular a raiz quadrada de um número.

As operações básicas (soma, subtração, multiplicação e divisão) utilizarão o conceito de passagem de parâmetros por valor. Já o procedimento raiz quadrada utilizará a passagem de parâmetros por referência.

Observe atentamente o Código 9.

Código 9



```

algoritmo "calculadora"
var
    num1, num2 : real
procedimento somar( a,b : real )
var
    resultado: real
inicio
    resultado <- a+b
    escreval("Soma = ", resultado)
fimprocedimento
procedimento subtrair( a,b : real )
var
    resultado: real
inicio
    resultado <- a-b
    escreval("Subtração = ", resultado)
fimprocedimento
procedimento multiplicar( a,b : real )
var
    resultado: real
inicio
    resultado <- a*b
    escreval("Multiplicação = ", resultado)
fimprocedimento
procedimento dividir( a,b : real )
var
    resultado: real
inicio
    se ( b = 0 ) entao
        escreval("Divisão por zero")
    senao
        resultado <- a/b
        escreval("Divisão = ", resultado)
    fimse
fimprocedimento
procedimento raizquadrada(var a: real)
var
    resultado : real
inicio
    resultado <- raizq(a)
    a <- resultado
fimprocedimento
inicio
    escreva("Número 1: ")
    leia(num1)
    escreva("Número 2: ")
    leia(num2)
    somar(num1,num2)
    subtrair(num1,num2)
    multiplicar(num1,num2)
    dividir(num1,num2)
    raizquadrada(num1)

```



```
escreval("Raiz quadrada = ", num1)
finalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:

```
Número 1: 9
Número 2: 3
Soma = 12
Subtração = 6
Multiplicação = 27
Divisão = 3
Raiz quadrada = 3
```

Antes de prosseguir para o próximo tópico, teste sua aprendizagem a respeito dos assuntos estudados até aqui.

Compreendidos os procedimentos, vamos entender como as funções podem modular um algoritmo.



3. Funções

As funções também são um mecanismo para a modularização de algoritmos. A principal característica de uma função é que ela **sempre retorna um valor** para o bloco principal do algoritmo.

Vimos, no tópico anterior, que os procedimentos são utilizados para executar um conjunto de instruções, mas não retornam valores para o bloco principal (exceto se utilizarmos a passagem de parâmetros por referência).

Atenção!

As funções sempre retornam informações. De modo geral, podemos pensar nas funções como algo que executa uma determinada tarefa e retorna o resultado da execução.

Observe, a seguir, a sintaxe para a declaração de uma função:

```
funcao <nome_da_funcao> (<parametros>): <tipo_retornado>

inicio

    {bloco de instruções}

fimfuncao
```

Veja que, além de definir o nome da função e a lista de parâmetros, é necessário definir qual tipo de dados será retornado pela função. Por exemplo: uma função que executa cálculos com números reais deve retornar um número do tipo real. Já uma função que manipula números inteiros pode retornar um número inteiro, bem como um número real.

Todas as funções devem, obrigatoriamente, retornar um valor. Assim, ao final do bloco interno de instruções da função, devemos utilizar a palavra reservada **retorne** para especificar qual valor será devolvido. Considere, por exemplo, uma função que some duas variáveis e retorna o resultado, conforme demonstrado a seguir:



```
funcao somar(a,b : real): real
inicio
    retorne (a+b)
fimfuncao
```

Note que a função somar recebe dois parâmetros (a e b) e retorna a soma desses valores. Como uma função sempre retorna um valor, devemos ter, no bloco principal do algoritmo, uma variável para receber esse valor, como, por exemplo:

```
resultado <- somar(num1,num2)
```

Podemos, ainda, exibir diretamente na tela o resultado da função:

```
escreva("Soma = ", somar(num1,num2) )
```

Lembre-se de que os conceitos estudados anteriormente sobre variáveis glo-

bais e variáveis locais também podem ser aplicados nas funções. Além disso, técnicas de passagem de parâmetros por valor e por referência também podem ser utilizadas.

Para ilustrar a utilização das funções, transformaremos o algoritmo **calculadora**, apresentado no Tópico 2, o qual foi elaborado com procedimentos, agora em uma versão baseada em funções. Observe:

Código 10



```

algoritmo "calculadora_funcao"
var
    num1, num2 : real
funcao somar( a,b : real ): real
inicio
    retorne (a+b)
fimfuncao
funcao subtrair( a,b : real ): real
inicio
    retorne (a-b)
fimfuncao
funcao multiplicar( a,b : real ): real
var
    resultado: real
inicio
    retorne (a*b)
fimfuncao
funcao dividir( a,b : real ): real
inicio
    se ( b = 0 ) entao
        retorne 0
    senao
        retorne (a/b)
    fimse
fimfuncao
funcao raizquadrada(a: real): real
inicio
    retorne raizq(a)
fimfuncao
inicio
    escreva("Número 1: ")
    leia(num1)
    escreva("Número 2: ")
    leia(num2)
    escreval("Soma = ", somar(num1,num2))
    escreval("Subtração = ", subtrair(num1,num2))
    escreval("Multiplicação = ", multiplicar(num1,num2))
    escreval("Divisão = ", dividir(num1,num2))
    escreval("Raiz quadrada = ", raizquadrada(num1))
finalgoritmo

```



Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:


```
Número 1: 9  
Número 2: 3  
Soma = 12  
Subtração = 6  
Multiplicação = 27  
Divisão = 3  
Raiz quadrada = 3
```

Neste momento, dê uma pausa em seus estudos e teste seus conhecimentos a respeito das funções.

No Código 11, elaboraremos outra nova versão do algoritmo **calculadora**, utilizando agora procedimentos e funções.

Neste exemplo, o usuário, além de informar os números para o cálculo, também deverá especificar qual o tipo de operação: **+** soma, **-** subtração, ***** multiplicação, **/** divisão ou **r** raiz quadrada.

Código 11



```

algoritmo "calculadora"
var
    num1, num2 : real
    operacao: caractere
procedimento calcular (a,b : inteiro; operacao: caractere)
inicio
    escolha(operacao)
    caso "+"
        escreval("Soma = ", somar(num1,num2))
    caso "-"
        escreval("Subtração = ", subtrair(num1,num2))
    caso "*"
        escreval("Multiplicação=", multiplicar(num1,num2))
    caso "/"
        escreval("Divisão = ", dividir(num1,num2))
    caso "r", "R"
        escreval("Raiz quadrada=", raizquadrada(num1))
    outrocaso
        escreva("Operação inválida.")
    fimescolha
fimprocedimento
funcao somar( a,b : real ): real
inicio
    retorne (a+b)
fimfuncao
funcao subtrair( a,b : real ): real
inicio
    retorne (a-b)
fimfuncao
funcao multiplicar( a,b : real ): real
var
    resultado: real
inicio
    retorne (a*b)
fimfuncao
funcao dividir( a,b : real ): real
inicio
    se ( b = 0 ) entao
        retorne 0
    senao
        retorne (a/b)
    fimse
fimfuncao
funcao raizquadrada(a: real): real
inicio
    retorne raizq(a)
fimfuncao
inicio
    escreva("Número 1: ")
    leia(num1)
    escreva("Número 2: ")
    leia(num2)

```



```
escreva("Operação [+,-,*,/,r]: ")
leia(operacao)
calcular(num1,num2,operacao)
finalgoritmo
```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:

```
Número 1: 9
Número 2: 3
Operação [+,-,*,/,r]: +
Soma = 12
```

No último tópico desta unidade, vamos conhecer uma importante técnica conhecida na linguagem de programação, a recursividade.

4. Recursividade



Como tivemos a oportunidade de aprender nos tópicos anteriores, podemos utilizar procedimentos e funções para modular os algoritmos. Todos os algoritmos que criamos até o momento são organizados de maneira estruturada, ou seja, as chamadas dos procedimentos e das funções ocorrem de forma disciplinada e hierárquica.

Entretanto, quando construímos algoritmos, podemos realizar chamadas das funções (e não dos procedimentos) de uma maneira alternativa. Essa outra maneira de chamar uma função faz com que ela seja capaz de chamar a si mesma. Essa técnica é conhecida por recursividade. Portanto, uma **função recursiva** é aquela que chama a si própria durante a execução do algoritmo.

Uma função recursiva é constituída por dois elementos principais:

1. **Parte básica:** é a parte do problema que a função sabe resolver.
2. **Parte recursiva:** é a parte do problema que a função não sabe resolver. Essa parte é parecida com o problema original, porém mais simples. Como a parte recursiva "parece" com o problema original, a função cha-

ma uma nova cópia de si mesma para resolver o problema menor. Essa chamada é denominada "recursiva".

Vejamos um exemplo: o fatorial de um número **n**, inteiro e não negativo, é escrito como **n!** e pronunciado como **fatorial de n**. O fatorial do número zero, ou seja, **0!**, é igual a 1 (um). Da mesma forma, **1!** é igual a 1 (um). Desse modo, para os demais valores, temos:

- a) $2! = 2$
- b) $3! = 6$
- c) $4! = 24$
- d) $5! = 120$
- e) ...

Assim, chegamos à seguinte fórmula para o cálculo do fatorial de qualquer número inteiro: **$N! = N * (N-1)!$** .

Podemos entender essa fórmula da seguinte maneira: o fatorial de um número inteiro (**N!**) é igual ao número (**N**) multiplicado pelo fatorial desse número menos 1 (**(N-1)!**). 

Seguindo os modelos de modularização que aprendemos até agora, podemos escrever um algoritmo para resolver esse problema da seguinte forma:

Código 12

```

algoritmo "fatorial"
var
    num: inteiro
funcao fatorial( n: inteiro ) : inteiro
var
    contador: inteiro
    resultado: inteiro
inicio
    resultado <- 1
    para contador de n ate 1 passo -1 faca
        resultado <- resultado * contador
    fimpara
    retorne resultado
fimfuncao
inicio
    escreva("Número: ")
    leia(num)
    escreva("Fatorial de ", num, " =", fatorial(num))
finalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:



```

Número: 4
Fatorial de 4 = 24

```

De maneira **recursiva**, esse algoritmo pode ser construído da forma a seguir:

Código 13

```

algoritmo "fatorial_13"
var
    num: inteiro
funcao fatorial( n: inteiro ) : inteiro
inicio
    se ( n <= 1 ) entao
        retorne 1
    senao
        retorne n * fatorial(n-1)
    fimse
fimfuncao
Inicio
    escreva("Número: ")
    leia(num)
    escreva("Fatorial de", num, " = ", fatorial(num))
fimalgoritmo

```

Para demonstrar o funcionamento do algoritmo, veja o resultado da execução, a seguir.

Resultado da execução do algoritmo:

```

Número: 4
Fatorial de 4 = 24

```



Neste momento, é essencial que você se autoavalie e verifique se compreendeu, de fato, a função recursiva. Para isso, responda à questão a seguir:

No algoritmo recursivo presente no Código 13, as Linhas “**se (n <= 1) entao**” e “**retorne 1**” definem a parte básica da função, ou seja, a parte que ela sabe resolver. Em contrapartida, a Linha “**retorne n * fatorial(n-1)**” determina a parte recursiva, na qual, em vez de resolver o problema, é realizada uma chamada recursiva da função fatorial.



Que tal aprofundar seus estudos a respeito da programação modular?

Para isso, realize as leituras indicadas na sequência.

- Capítulo 11 – *Módulos de Programas*, da obra *Linguagem e lógica de*

programação (ALVEZ, 2014).

- Capítulo 12 – *Do algoritmo para uma Linguagem de programação*, da obra *Linguagem e lógica de programação* (ALVEZ, 2014).
- Capítulo 10 – *Subprogramas*, da obra *Algoritmos – Lógica para Desenvolvimento de Programação de Computadores* (MANZANO; OLIVEIRA, 2019). Estas obras estão disponíveis na Minha Biblioteca.

5. Considerações finais

Chegamos ao final do estudo desta unidade, no qual tivemos a oportunidade de aprender como modularizar algoritmos. Estudamos também os principais conceitos relacionados às sub-rotinas, tais como: variáveis globais e variáveis locais; passagem de parâmetros.

Por fim, descobrimos como utilizar funções e procedimentos na construção de algoritmos. Com isso, concluímos o estudo de **Algoritmos e Programação**. Por meio deste material, você teve a possibilidade de conhecer os principais conceitos relacionados à elaboração de algoritmos e pôde visualizar diversos exemplos práticos.

Sucesso e até a próxima!



Créditos

Currículos dos autores

Luciano Bérghamo é mestre em Engenharia de Produção com pesquisa em *Business Intelligence* pela Universidade de Araraquara. Possui pós-graduação (MBA) em Tecnologia da Informação voltada para a Gestão Estratégica de Negócios pela Fundação Getúlio Vargas (FGV), especialização em Banco de Dados e graduação em Tecnologia em Processamento de Dados pela Universidade de Franca. Lecionou como tutor presencial nos cursos da Universidade Virtual do Estado de São Paulo (UNIVESP) entre 2017 e 2018. Atualmente, é docente concursado pela Faculdade de Tecnologia do Estado de São Paulo (FATEC) nas unidades de Franca, Mococa e Sertãozinho desde 2011. Também, é coordenador e professor de cursos (EaD e presencial) no Claretiano – Centro Universitário desde 2005.



Para conhecer a vida acadêmica e profissional do professor Me. Luciano Bérghamo, acesse seu *Currículo Lattes* (<http://lattes.cnpq.br/9366315455561553>).

Rodrigo de Oliveira Plotze é doutor na área de Computação Gráfica e Processamento de Imagens pelo Instituto de Ciências Matemáticas e de Computação da USP (São Carlos). Desenvolve pesquisas na área de Visão Computacional aplicada à Biologia e à Medicina, na qual tem diversos trabalhos publicados em periódicos nacionais e internacionais. Atua, profissionalmente,

como docente nos cursos de pós-graduação em Desenvolvimento de Software (Java e .NET) no Claretiano – Centro Universitário e, também, em cursos de graduação em Ciência da Computação, Engenharia da Computação e Sistemas de Informações em outras instituições de ensino superior.

Para conhecer a vida acadêmica e profissional do professor Dr. Rodrigo de Oliveira Plotze, acesse seu *Currículo Lattes* (<http://lattes.cnpq.br/4956606071703925>).

Versão: ago. 2022

DADOS INTERNACIONAIS DE CATALOGAÇÃO NA PUBLICAÇÃO (CIP)
(Câmara Brasileira do Livro, SP, Brasil)



005.1 B433a

Bérgamo, Luciano

Algoritmos e programação / Luciano Bérgamo, Rodrigo de Oliveira Plotze – 2. ed. – Batatais, SP: Claretiano, 2022.

ISBN: 978-85-8377-691-8

1. Programação de computadores. 2. Algoritmos. 3. Variáveis. 4. Comandos condicionais. 5. Estrutura de repetição. I. Plotze, Rodrigo de Oliveira. II. Algoritmos e programação.

CDD 005.1

Bibliotecária

Ana Carolina Guimarães – CRB8 - 9344

Revisor Técnico

Renato de Oliveira Violin

Equipe Editorial

Gerente do Material Didático

Rodrigo Ferreira Daverni

Preparação

Aline de Fátima Guedes

Ana Laura Mulati Rodrigues

Camila Maria Nardi Matos

Carolina de Andrade Baviera

Cátia Ap. Ribeiro

Lidiane Maria Magalini

Luciana A. Mani Adami

Luciana S. Sançana de Melo

Patrícia Alves Veronez Montera

Simone Rodrigues de Oliveira

Vanessa Ito Nazar



Videoaula

André Luís Menari Pereira

Bruna Giovanaz Bulgarelli

Fernanda Maestu

Guilherme Eduardo de Oliveira

Luis Gustavo Millan

Luiz Thomaz Nunes

Marilene Baviera

Nicholas Ferrarezi Balan

Sidiney Canelli Neto

Revisão

Eduardo Henrique Marinheiro

Filipi Andrade de Deus Silveira

Taila Poliana Cintra Rosa

Vanessa Vergani Machado

Projeto gráfico e diagramação

Joice Cristina Micai da Silva

Lúcia Maria de Sousa Ferrão

Luis Antônio Guimarães Toloí

Raphael Fantacini de Oliveira

Tamires Botta Murakami

